

# CSE/ECE 470: Computer Systems Architecture

*Pipelining Review*

Spring 2022

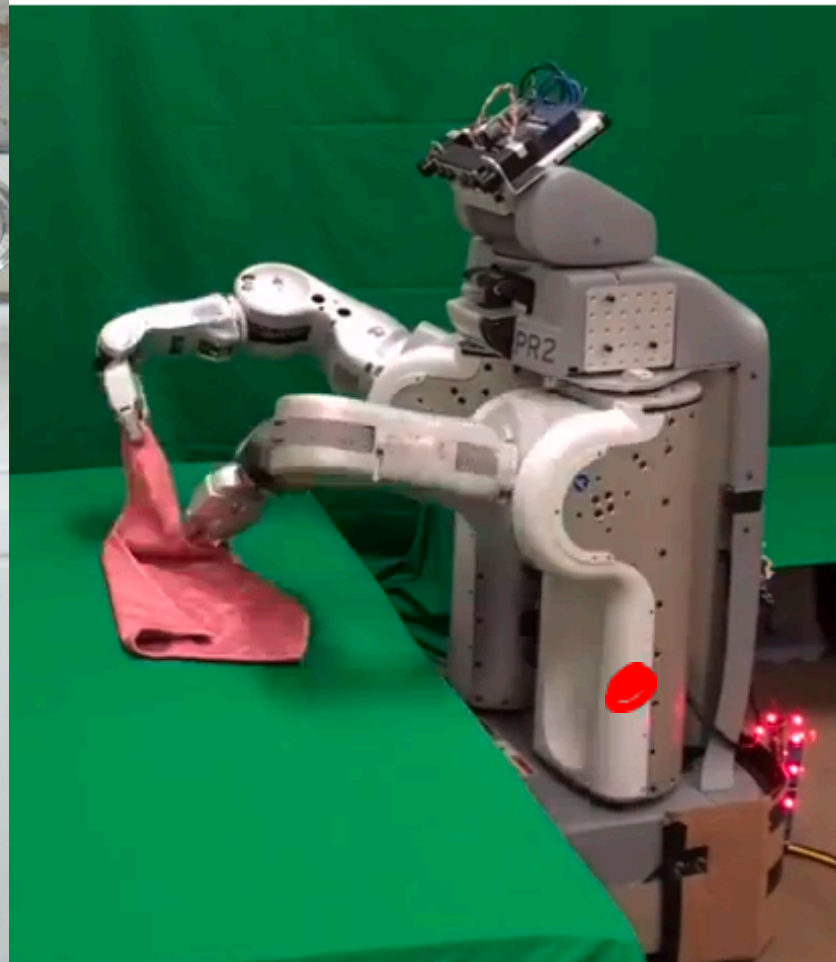
Luis Ceze (Instructor)

(based on slides lifted from friends at UPenn, UIUC, UW, MIT).

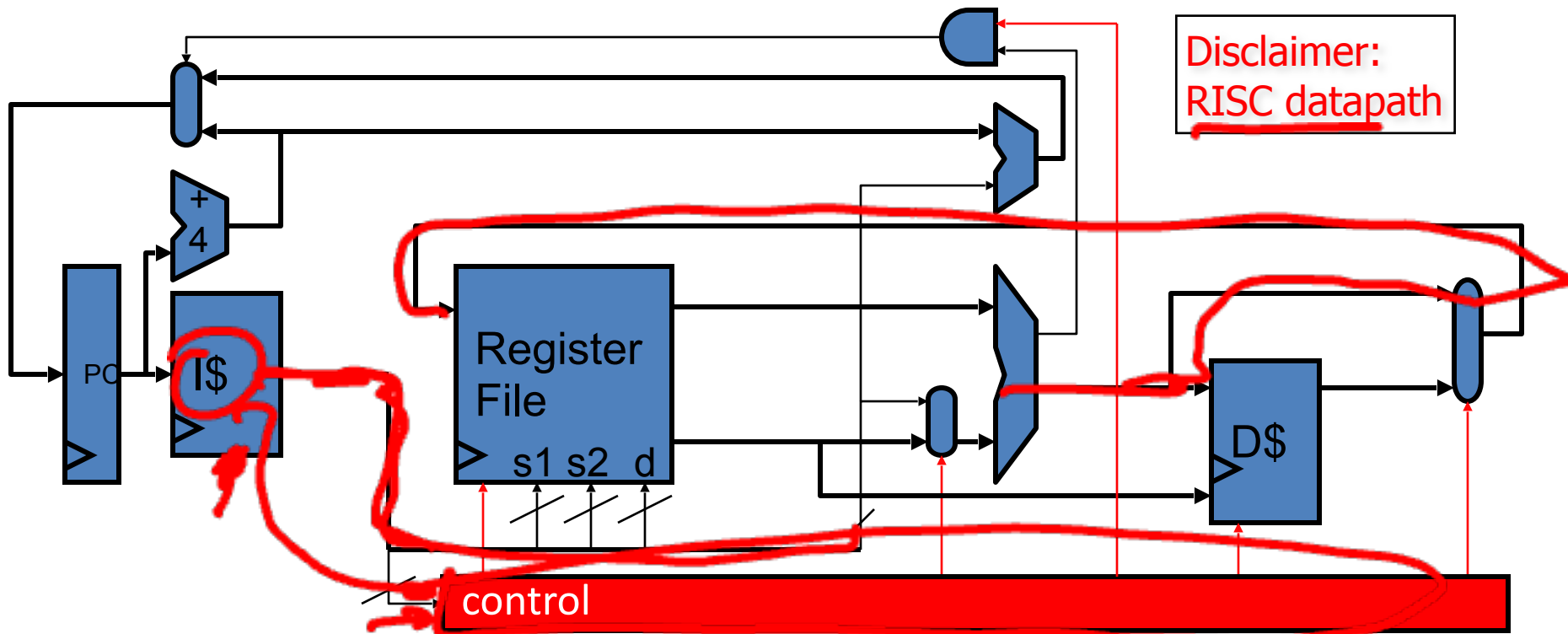
# (Scalar In-Order) Pipelining

- Basic Pipelining
  - Pipeline control
- Data Hazards
  - Software interlocks and scheduling
  - Hardware interlocks and stalling
  - Bypassing
- Control Hazards
  - Branch prediction

# The eternal pipelining metaphor

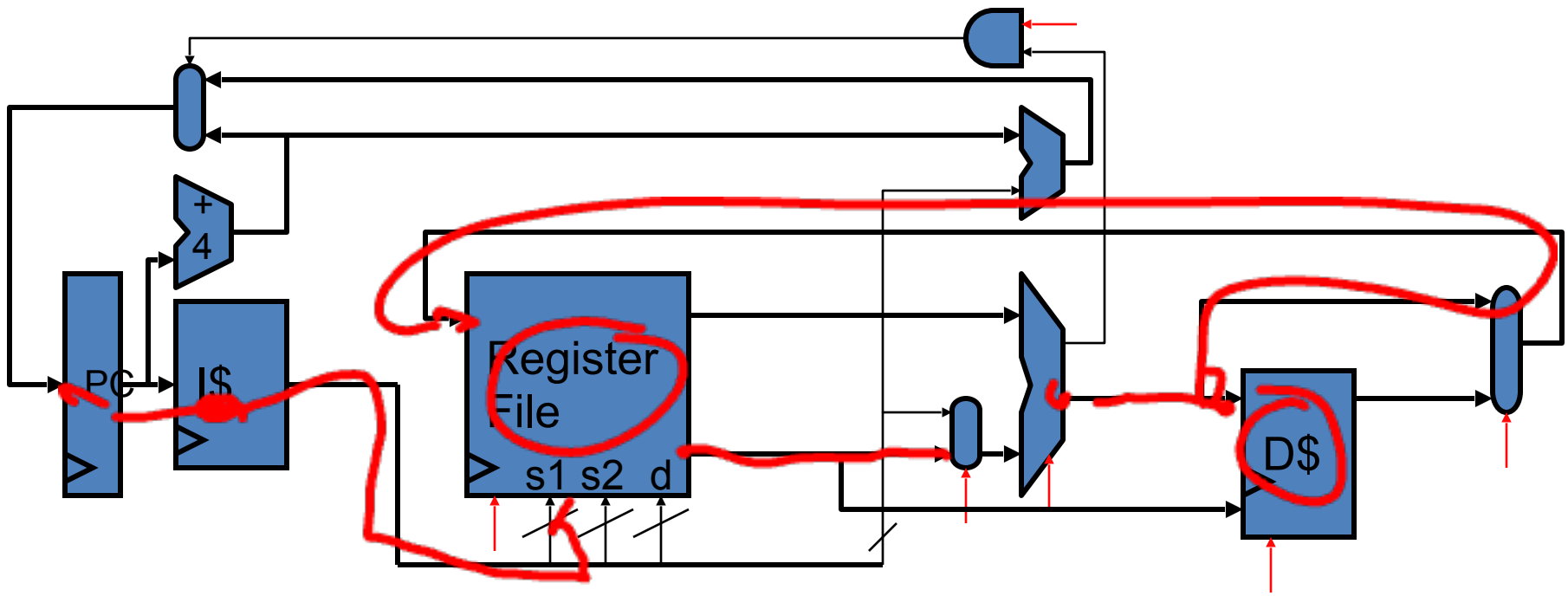


# Datapath and Control



- **Datapath:** implements execute portion of fetch/exec. loop
  - Functional units (ALUs), registers, memory interface
- **Control:** implements decode portion of fetch/execute loop
  - Mux selectors, write enable signals regulate flow of data in datapath
  - Part of decode involves translating insn opcode into control signals

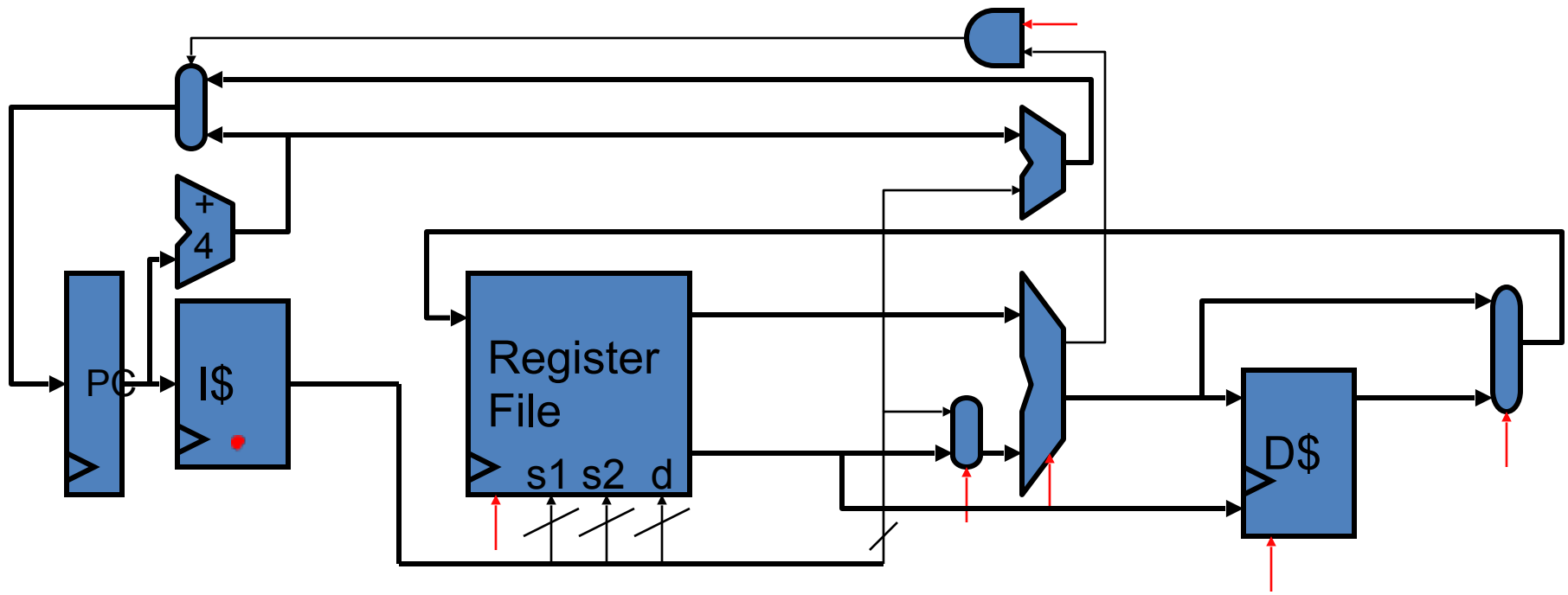
# Single-Cycle Datapath



- **Single-cycle datapath:** true “atomic” VonNeuman loop
  - Fetch, decode, execute one complete insn every cycle
  - *What is the CPI? What happens to the clock cycle time?*

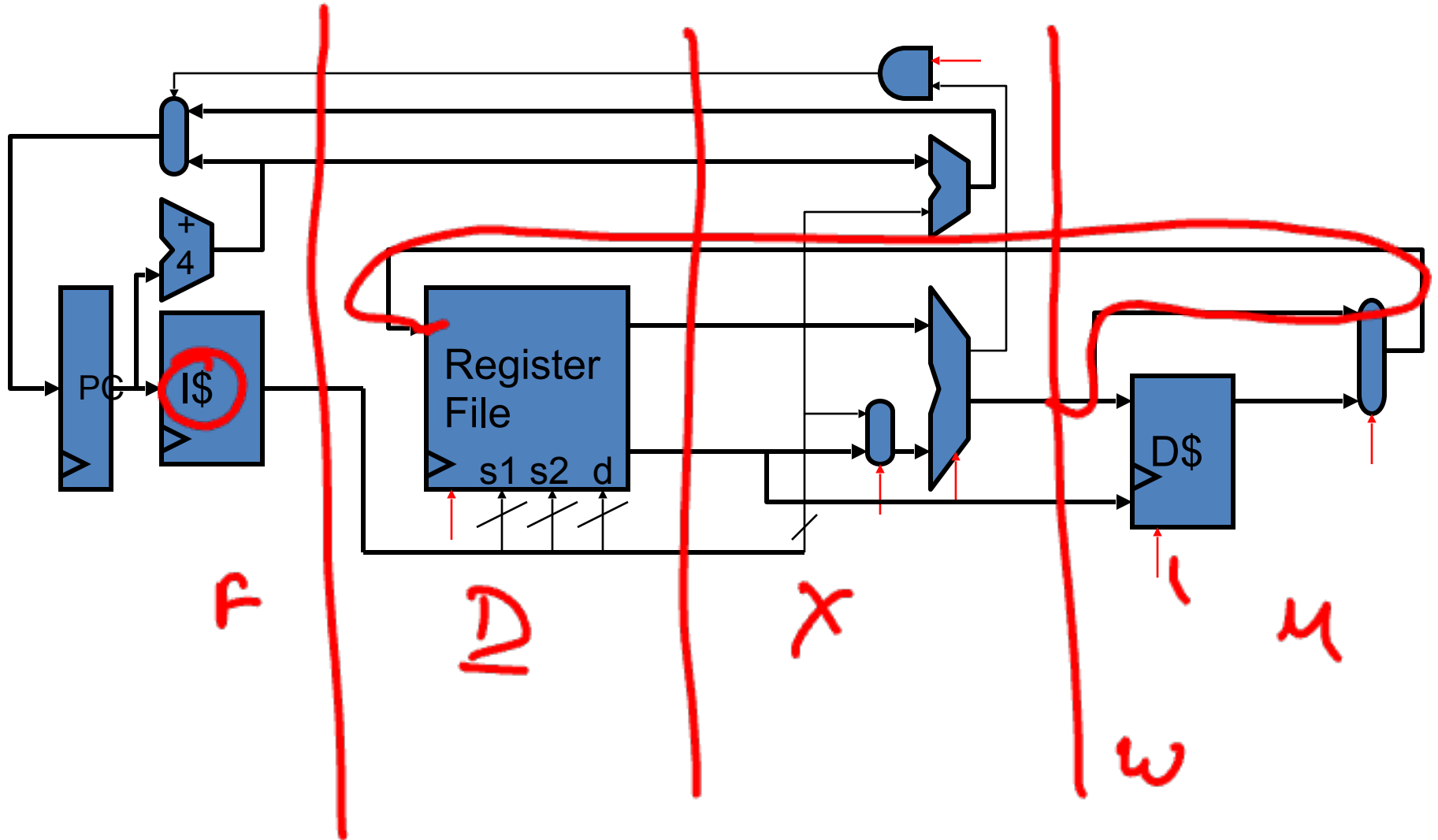
1

# Single-Cycle Datapath

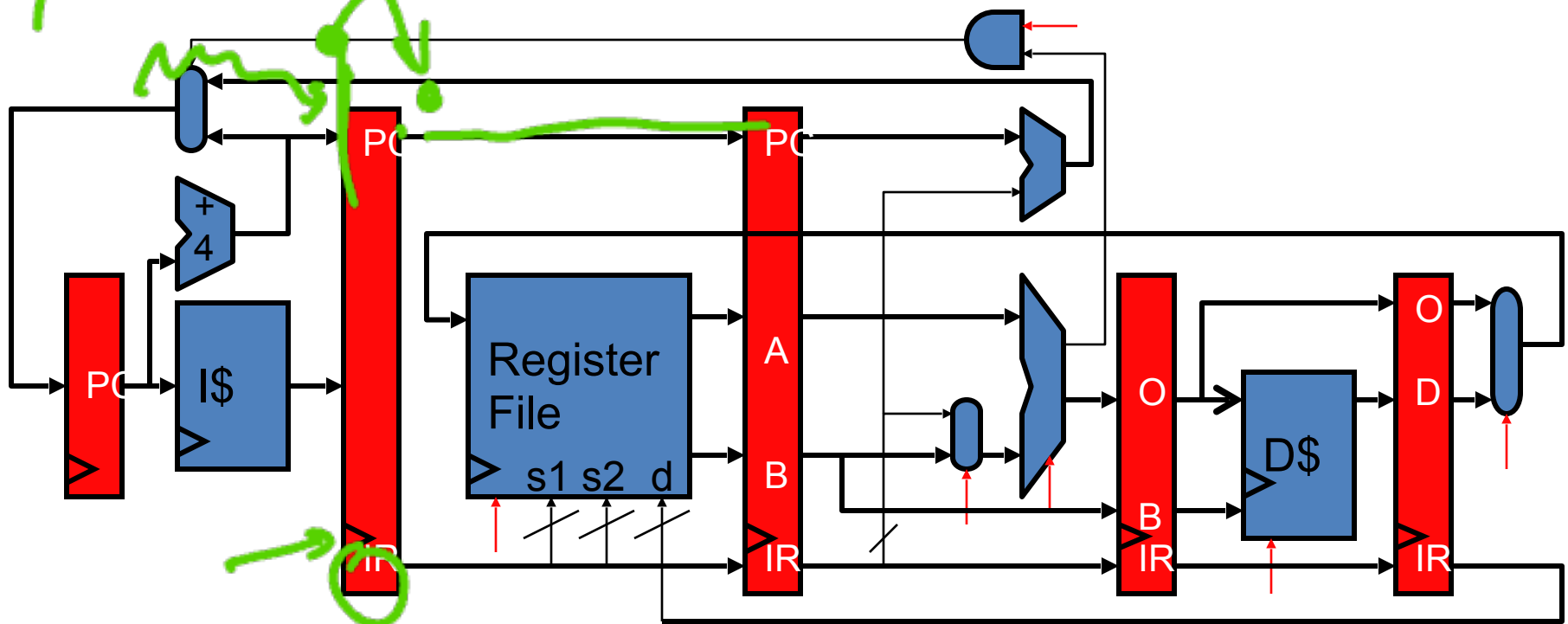


- **Single-cycle datapath:** true “atomic” VonNeuman loop
  - Fetch, decode, execute one complete insn every cycle
  - **“Hardwired control”:** opcode to control signals ROM
  - + Low CPI: 1 by definition
  - Long clock period: to accommodate **longest** insn
- Does all this work need to be done in one shot?

Can we just chop this up?

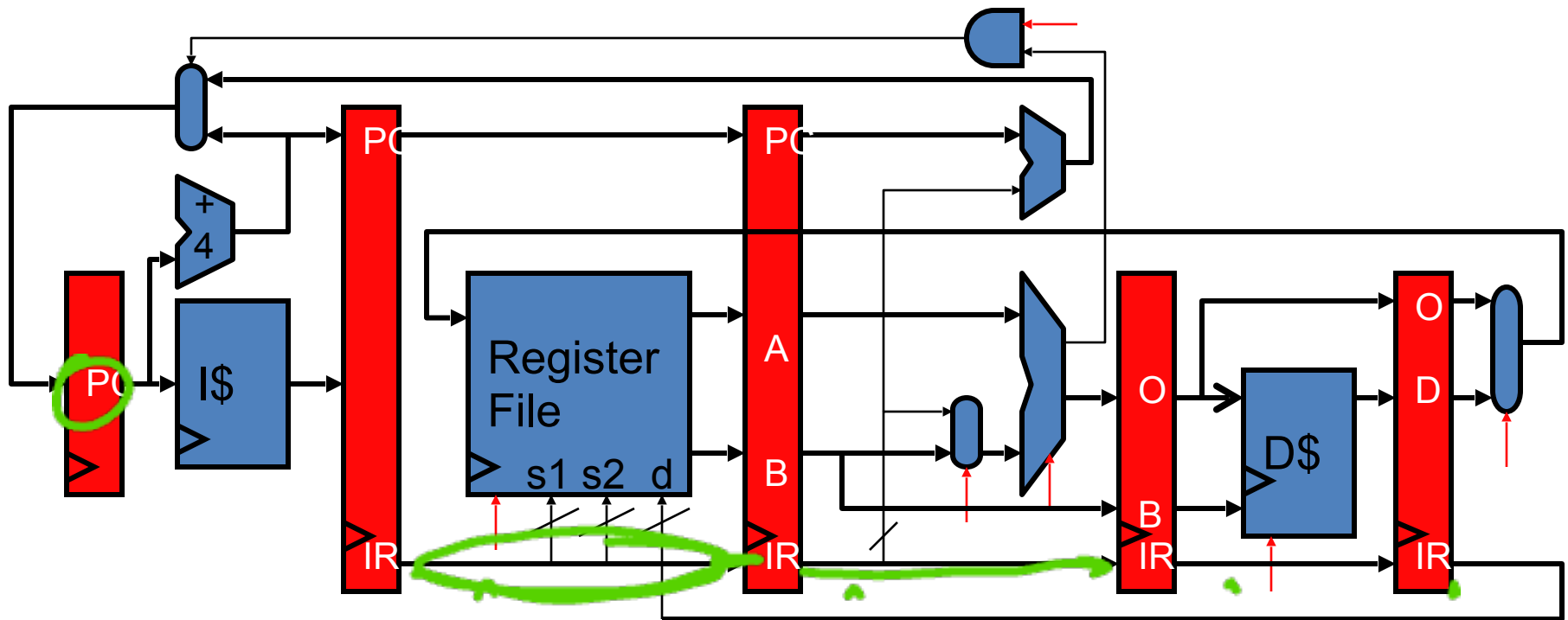


# 5 Stage Pipelined Datapath



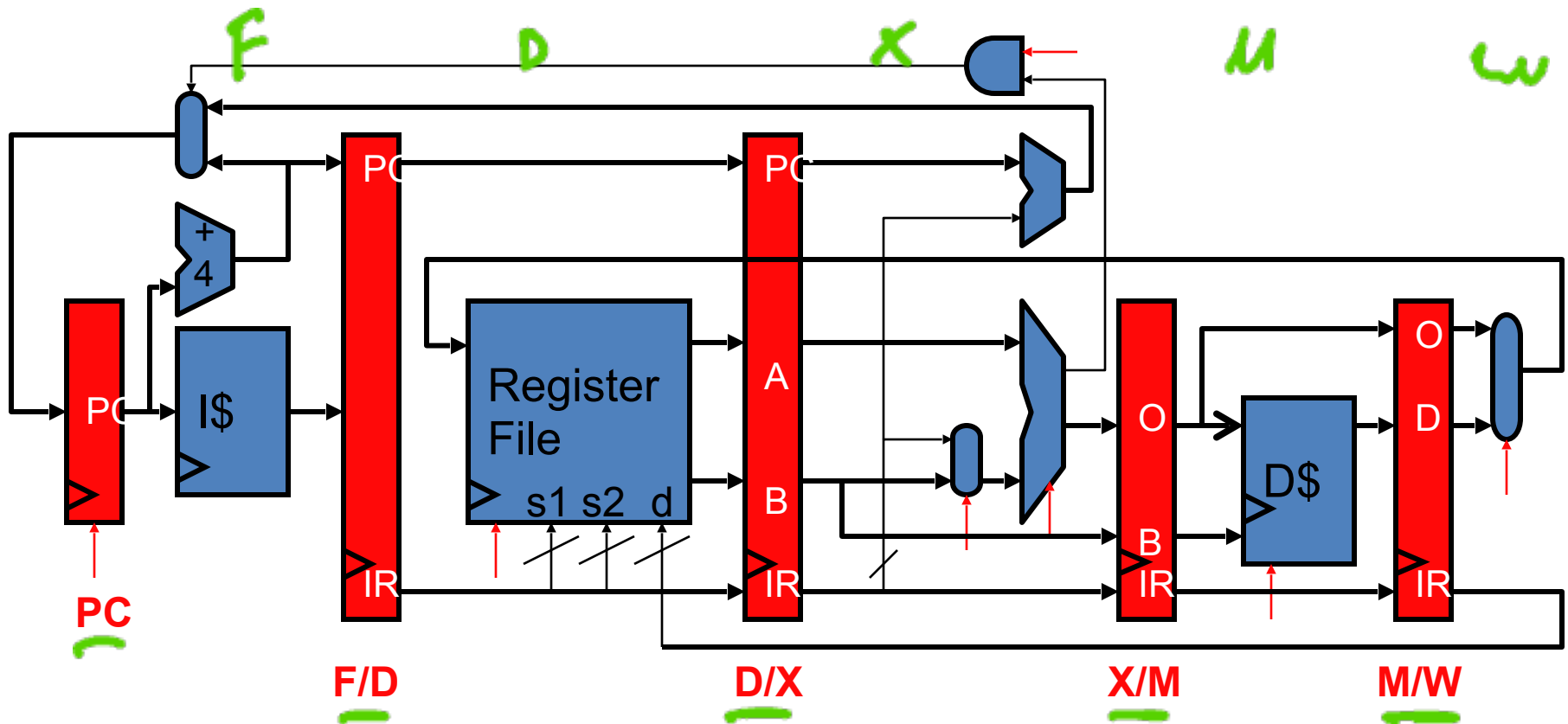
- Temporary values (PC,IR,A,B,O,D) re-latched every stage
  - Why?

# 5 Stage Pipelined Datapath



- Temporary values (PC,IR,A,B,O,D) re-latched every stage
  - Why? 5 insns may be in pipeline at once with different PCs
  - **Pipelined control**: Control signals themselves pipelined

# Pipeline Terminology



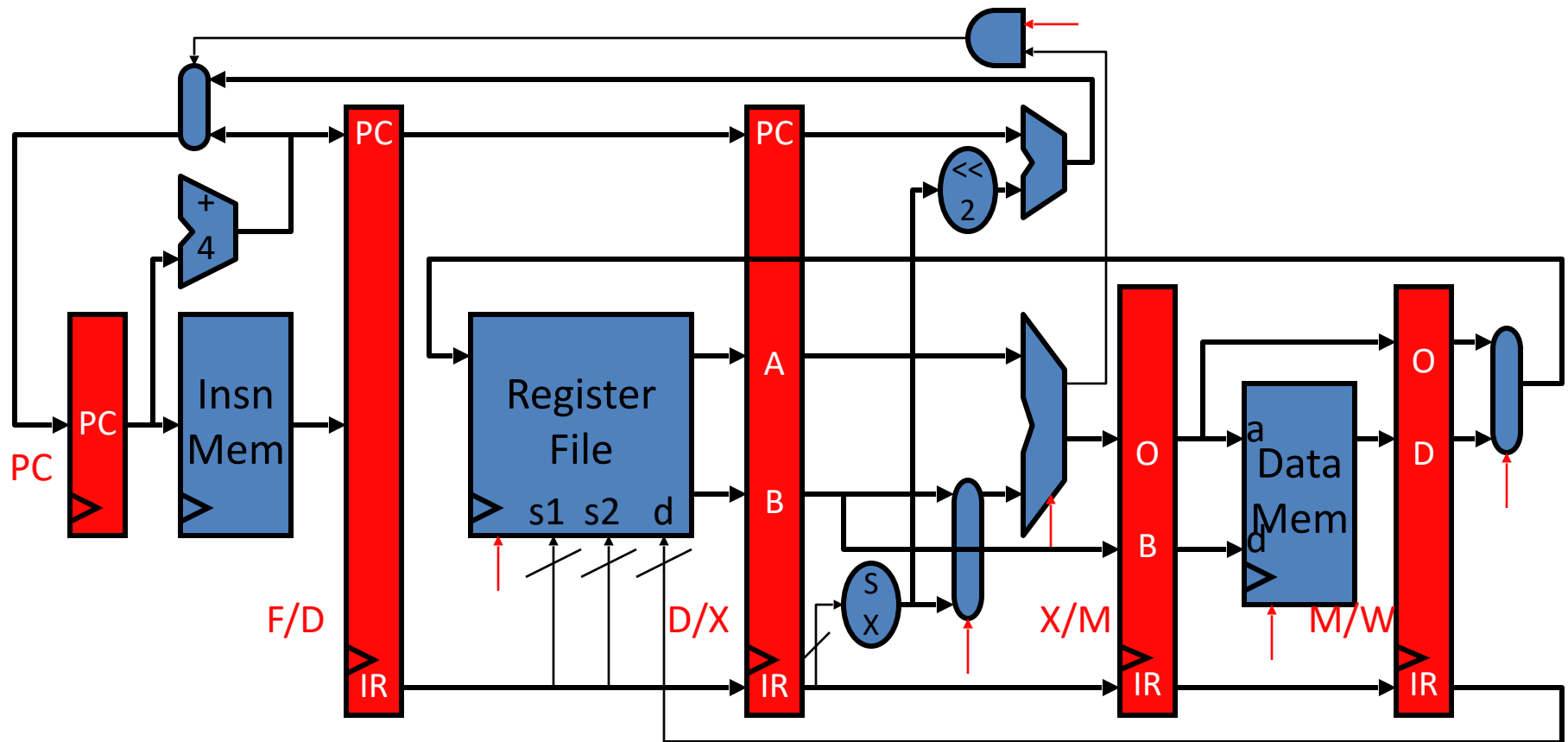
- Five stage: **F**etch, **D**ecode, e**X**ecute, **M**emory, **W**riteback
  - Nothing magical about the number 5 (Core i7 has 29+ stages)
- Latches (pipeline registers) named by stages they separate
  - **PC**, **F/D**, **D/X**, **X/M**, **M/W**

# More Terminology & Foreshadowing

- **Scalar pipeline**: one instruction per stage per cycle
  - Alternative: “superscalar”, or multiple insns per cycle (later)
- **In-order pipeline**: instruction in *execute* stage in program order
  - Alternative: “out-of-order” (later)
- **Pipeline depth**: number of pipeline stages
  - Nothing magical about five
  - Trend has been to deeper pipelines

20

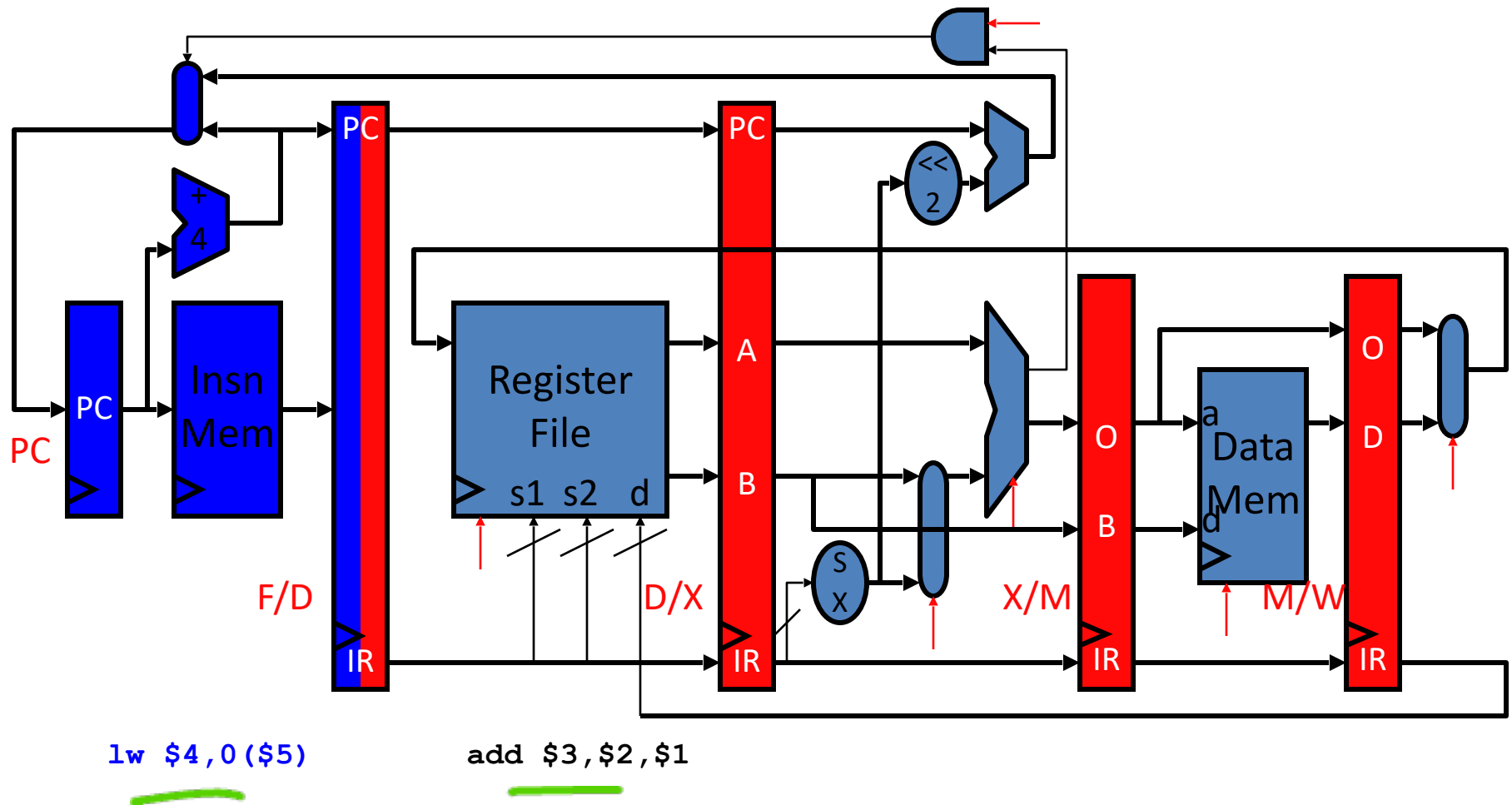
# Pipeline Example: Cycle 1



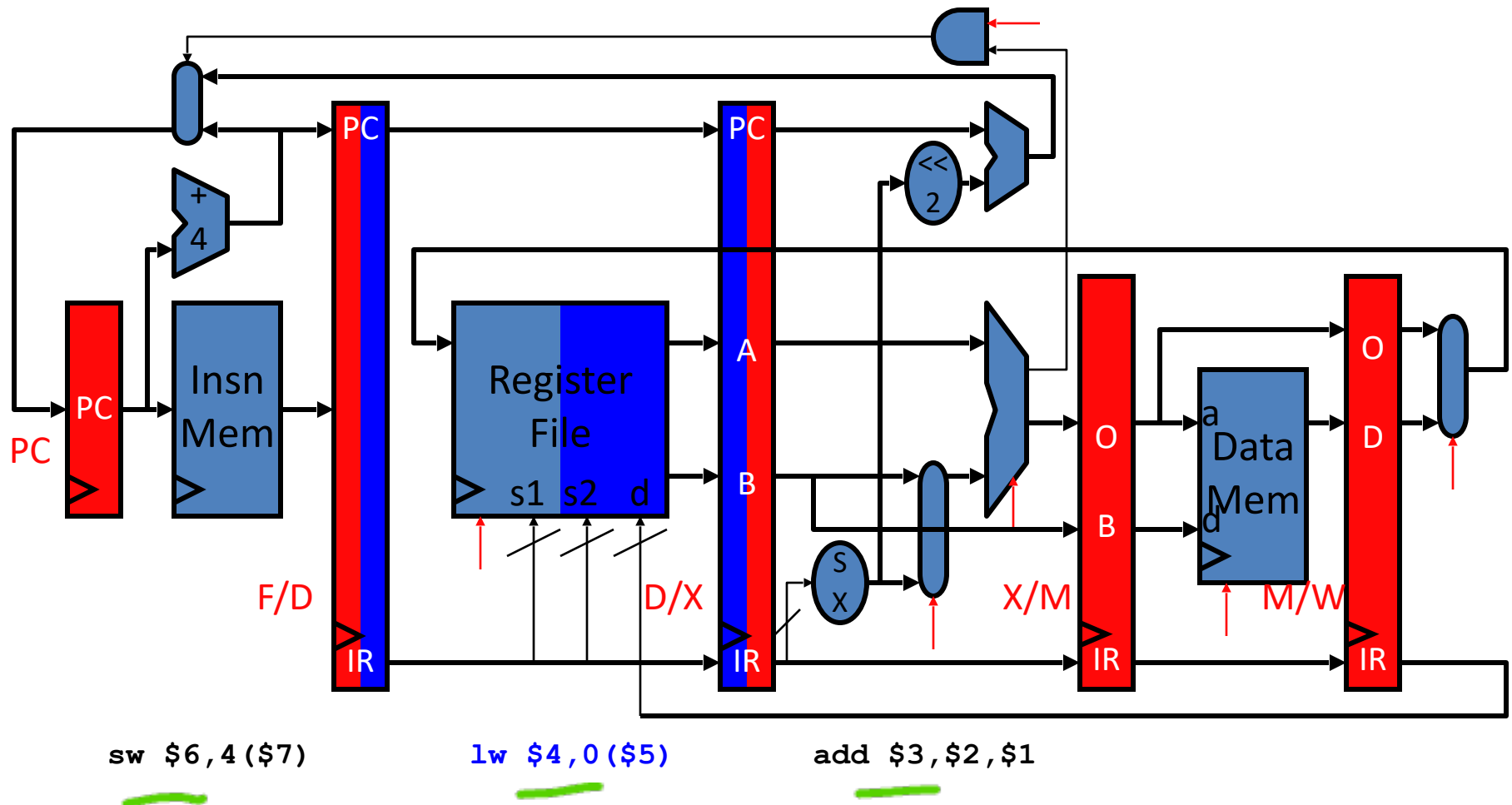
add \$3, \$2, \$1

- 3 instructions

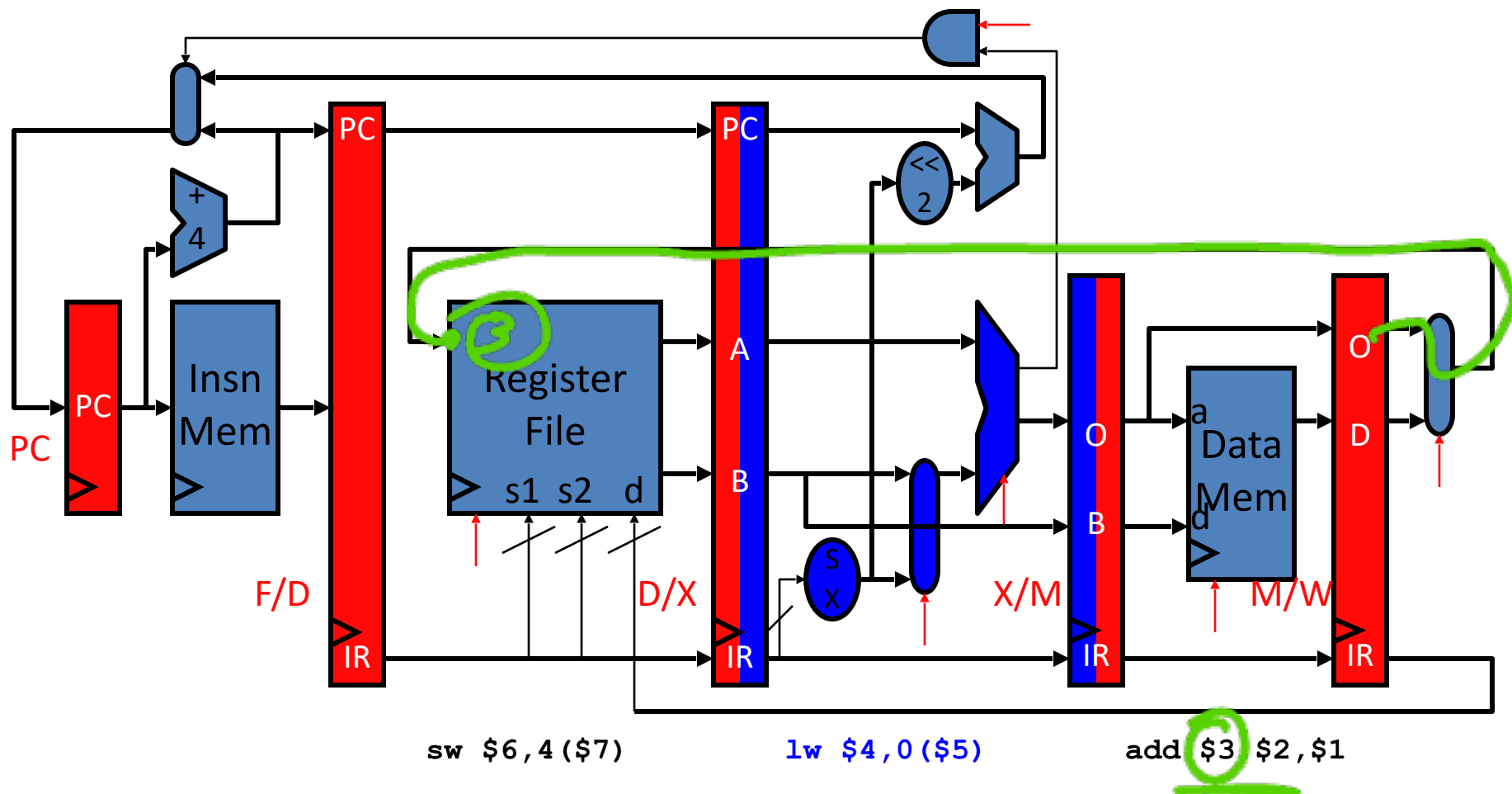
# Pipeline Example: Cycle 2



# Pipeline Example: Cycle 3

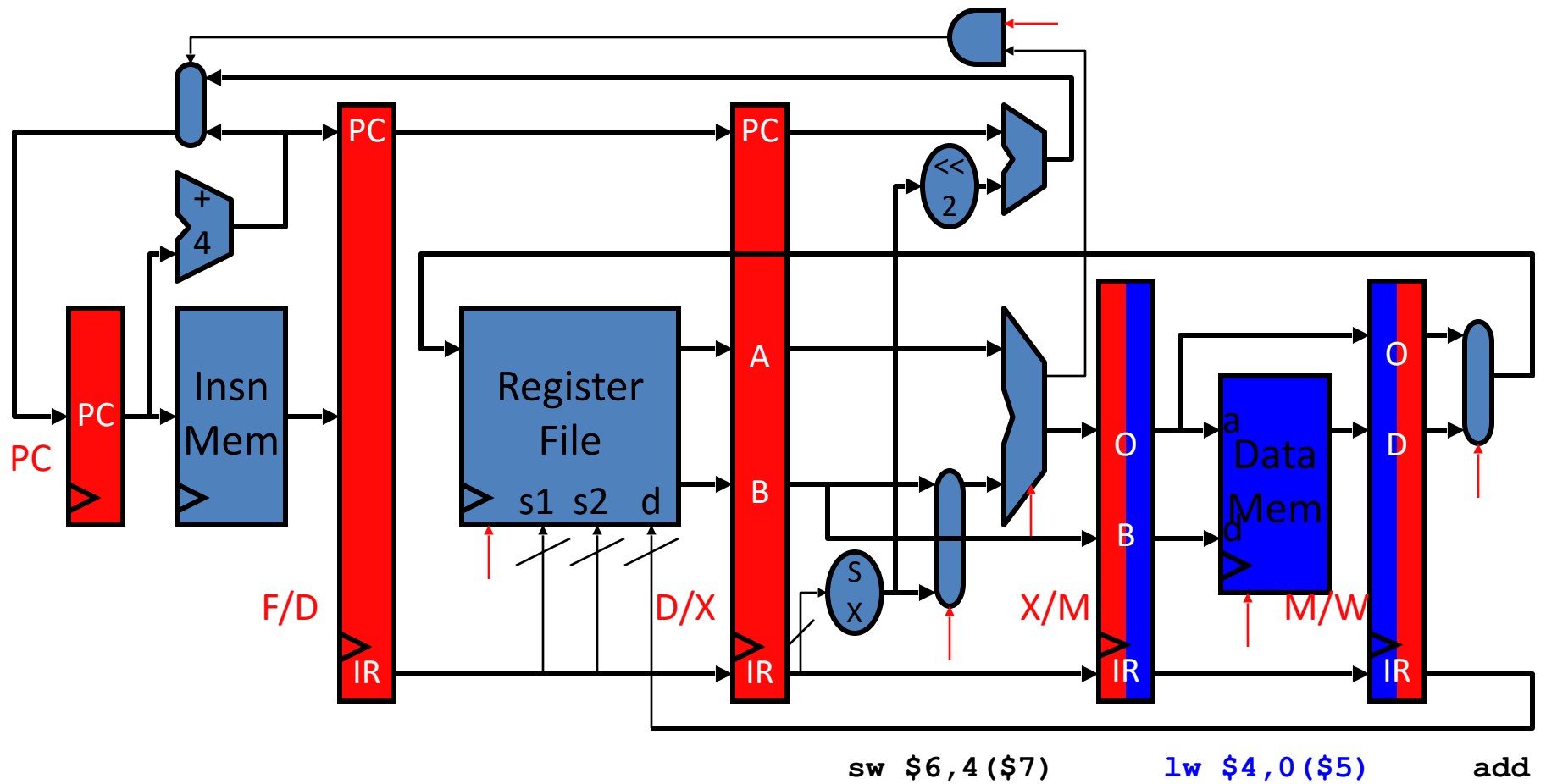


# Pipeline Example: Cycle 4

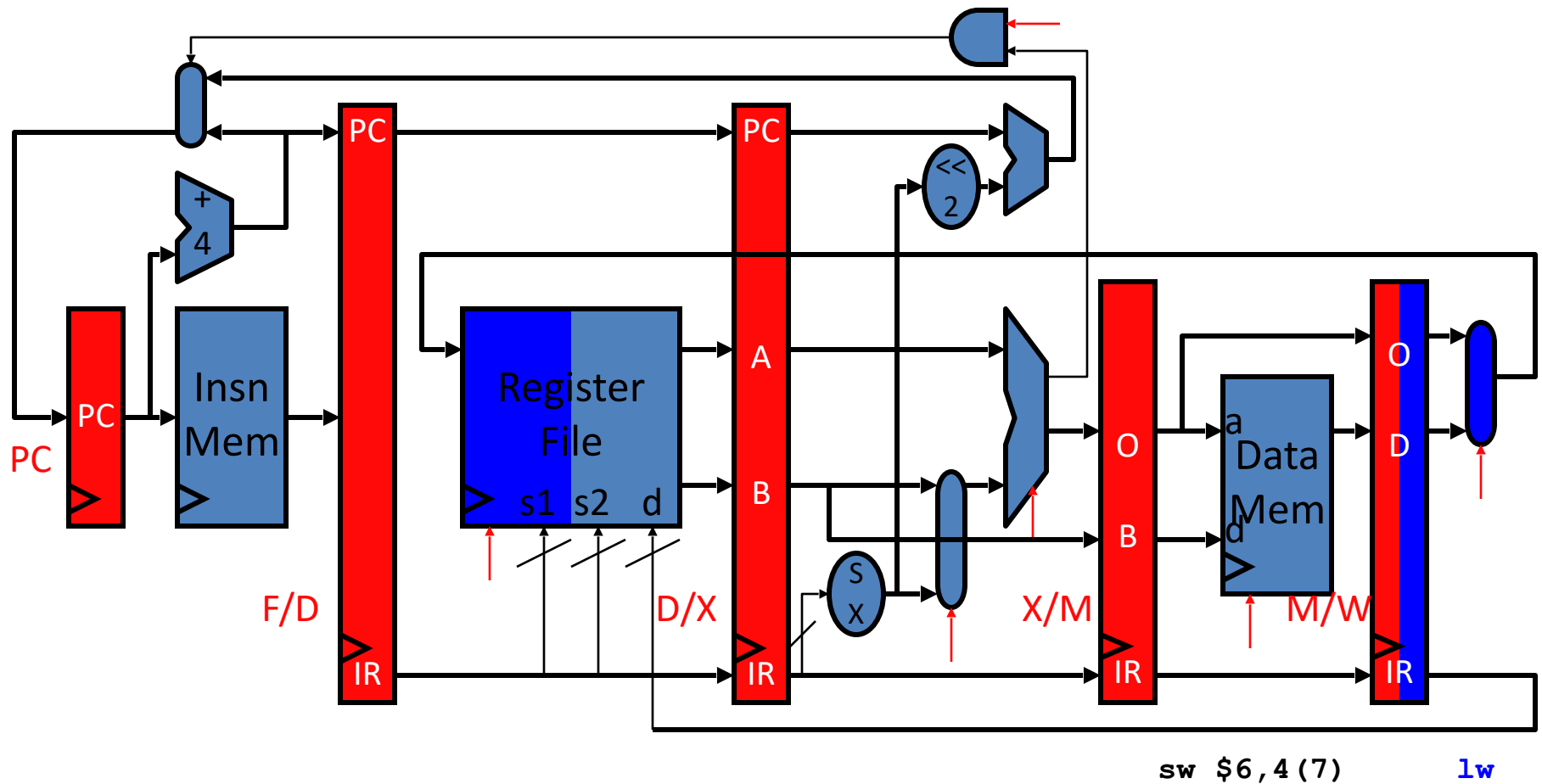


- 3 instructions

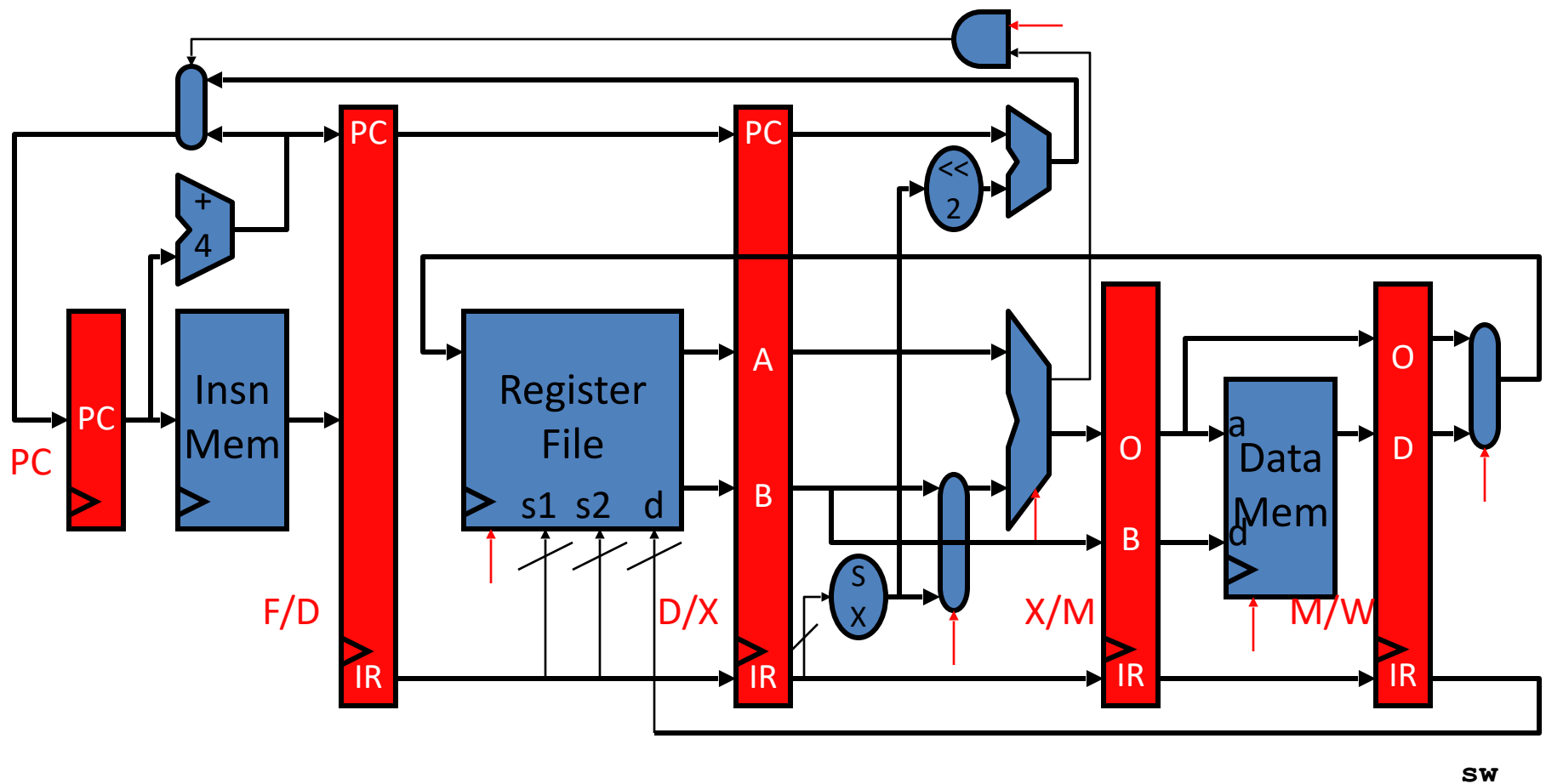
# Pipeline Example: Cycle 5



# Pipeline Example: Cycle 6

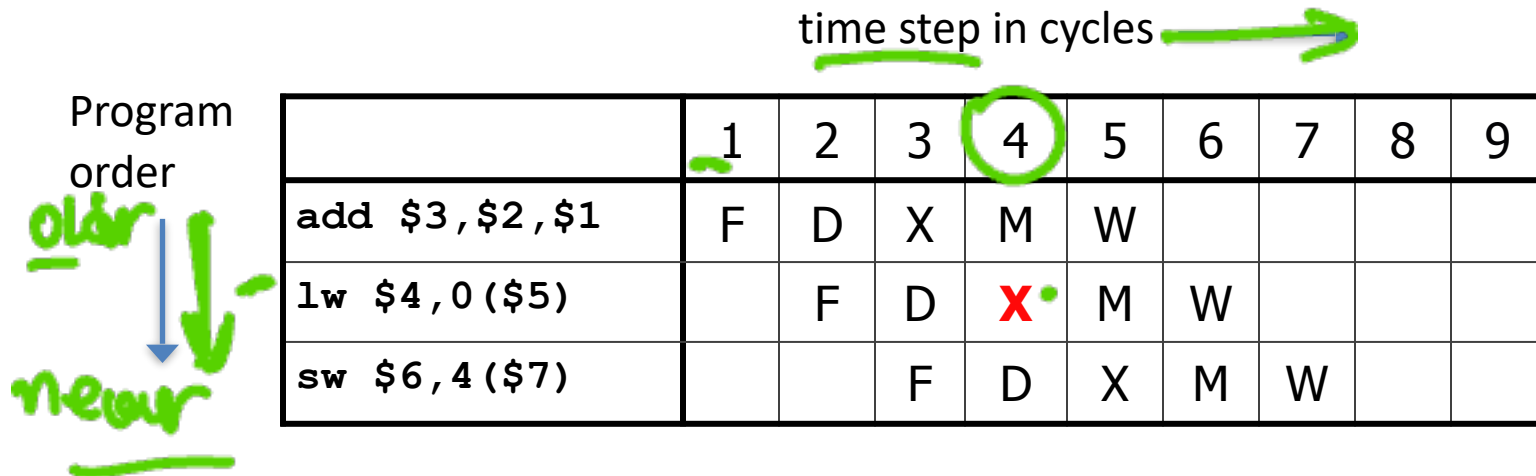


# Pipeline Example: Cycle 7



# Pipeline Diagram

- **Pipeline diagram:** shorthand for what we just saw
  - Across: cycles
  - Down: insns
  - Convention: **X** means `lw $4, 0($5)` finishes execute stage and writes into X/M latch at end of cycle 4



# Example Pipeline Perf. Calculation

- Single-cycle
  - Clock period = 50ns, CPI = 1
  - Performance = 50ns/insn
- 5-stage pipelined?
  - Clock period = 12ns (approx. (50ns / 5 stages) + overheads)
  - + CPI ? 1
  - + Performance? 12ns/insn

# Example Pipeline Perf. Calculation

- Single-cycle
  - Clock period = 50ns, CPI = 1
  - Performance = 50ns/insn
- 5-stage pipelined
  - Clock period = **12ns** (approx. (50ns / 5 stages) + overheads)
  - + CPI = **1** (each insn takes 5 cycles, but 1 completes each cycle)
  - + Performance = **12ns/insn**
  - Well actually ... CPI = 1 + some penalty for pipelining (next)
    - CPI = **1.5** (on average insn completes every 1.5 cycles)
    - Performance = **18ns/insn**
    - Latches add delay
    - Extra “bypassing” logic adds delay
    - Pipeline stages have different delays, clock period is max delay

# Q1: Why Is Pipeline *CPI*...

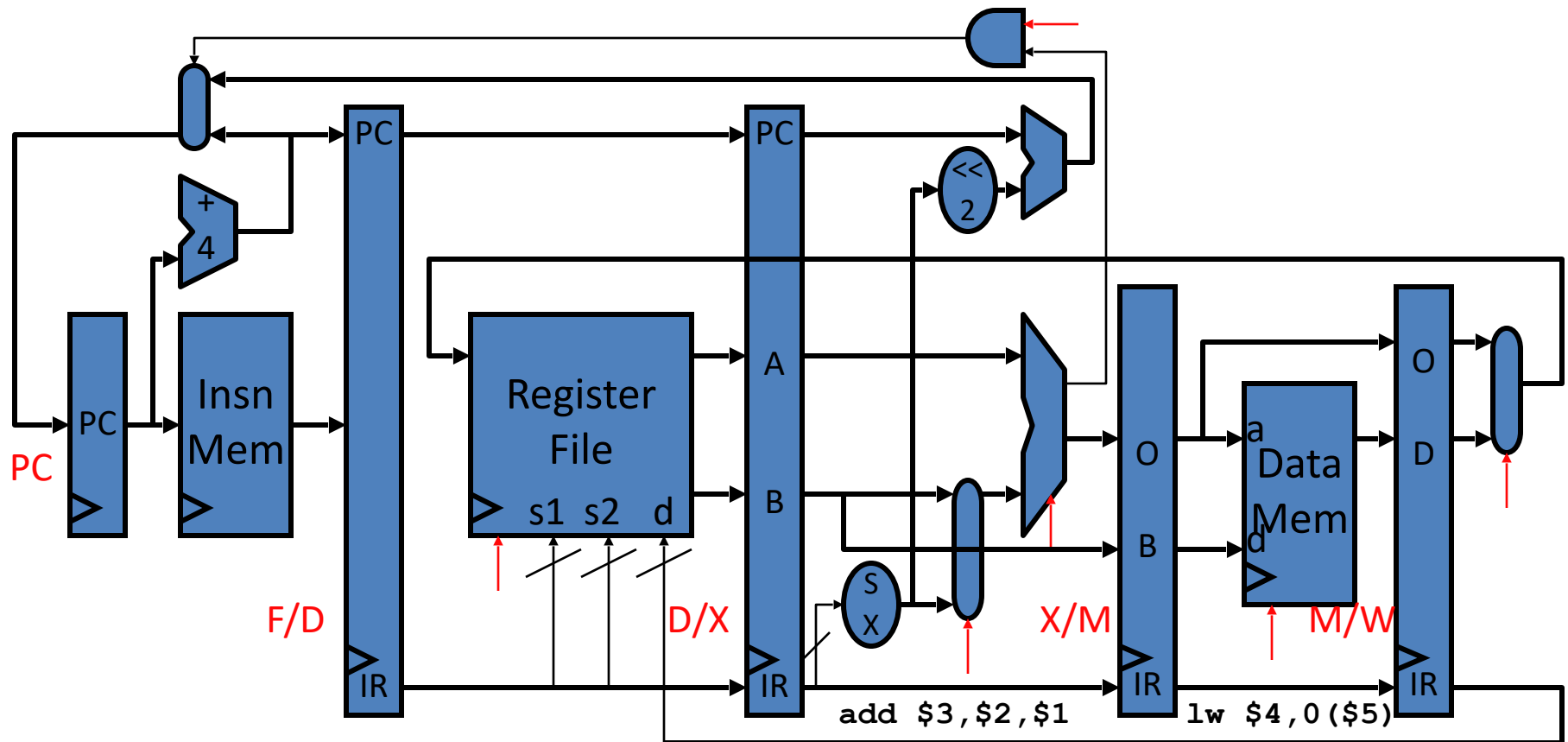
- ... > 1?
  - CPI for scalar in-order pipeline is 1 + stall penalties
  - Pipelining is not always smooth...
  - Stalls used to resolve hazards
    - Hazard: condition that jeopardizes pipeline flow
    - Stall: pipeline delay introduced to restore pipeline flow
- Calculating pipeline CPI
  - Frequency of stall \* stall cycles
  - Penalties add
  - $1 + \text{stall-freq}_1 * \text{stall-cyc}_1 + \text{stall-freq}_2 * \text{stall-cyc}_2 + \dots$
- Correctness/performance/make common case fast (MCCF)
  - Long penalties OK if they happen rarely, e.g.,  $1 + 0.01 * 10 = 1.1$
  - Stalls also have implications for ideal number of pipeline stages

What can go *wrong* in pipelined execution?

# Dependences and Hazards

- **Dependence**: relationship between two insns
  - **Data**: two insns use same storage location R, M
  - **Control**: one insn affects whether another executes at all
  - Not a bad thing, programs would be boring without them
  - Enforced by making older insn go before younger one
    - Happens naturally in single-/multi-cycle designs
    - But not in a pipeline
- **Hazard**: dependence & possibility of wrong insn order
  - Effects of wrong insn order cannot be externally visible
    - **Stall**: for order by keeping younger insn in same stage
  - Hazards are a bad thing: stalls reduce performance

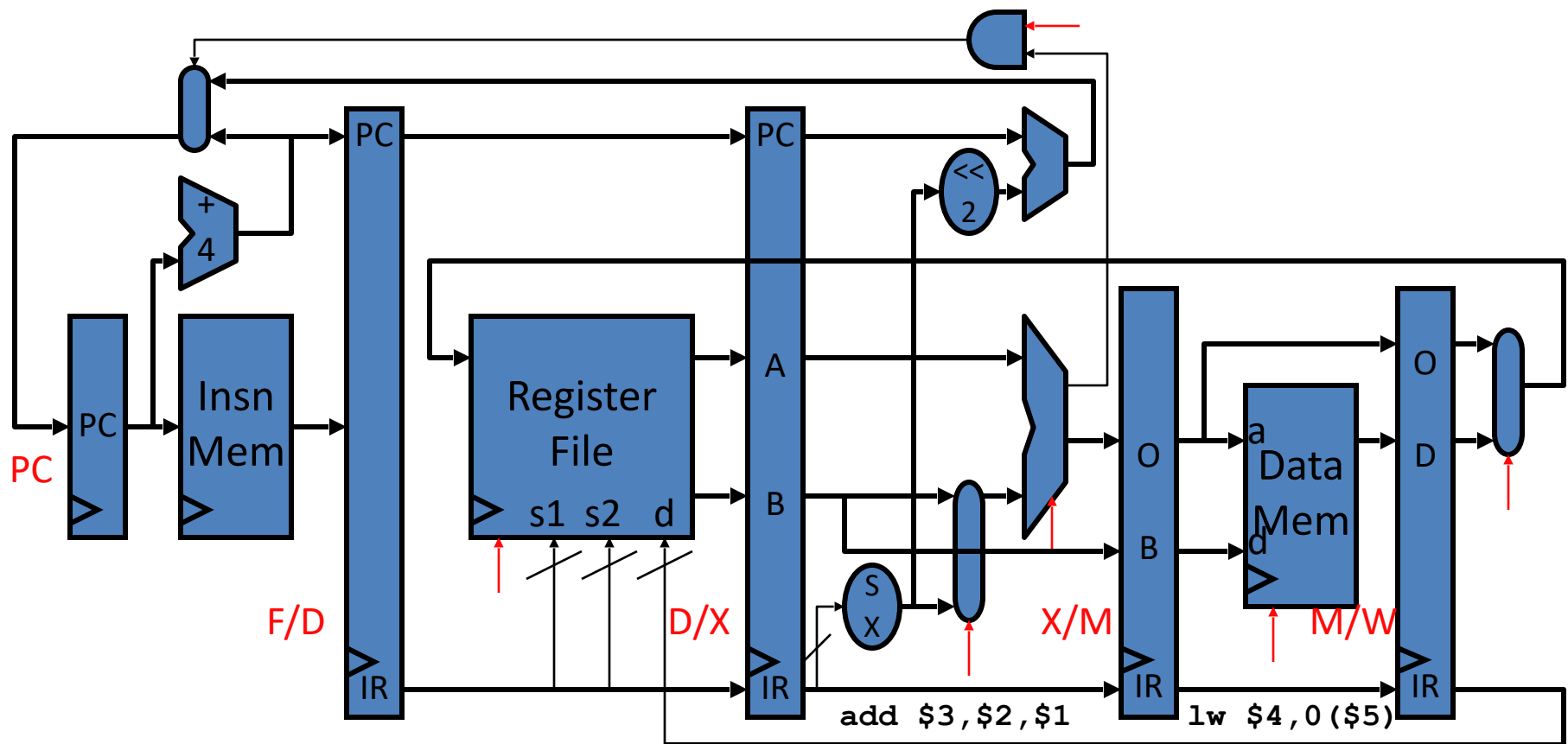
# Why Does Every Insn Take 5 Cycles?



- Could/should we allow **add** to skip M and go to W?

**no**

# Why Does Every Insn Take 5 Cycles?



- Could/should we allow **add** to skip M and go to W? No
  - It wouldn't help: peak fetch still only 1 insn per cycle
  - **Structural hazards:** **add** and **lw** would use same circuit

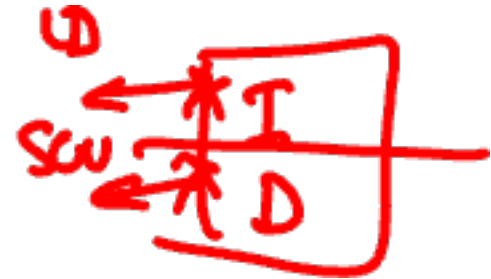
# Structural Hazards

- **Structural hazards** —
  - Two insns trying to use same circuit at same time
    - E.g., structural hazard on regfile write port
- **To fix structural hazards:** proper ISA/pipeline design *n/w*
  - Each insn uses every structure exactly once
  - For at most one cycle
- **Tolerate structure hazards** —
  - Add stall logic to stall pipeline when hazards would occur

# Example Structural Hazard

|              | 1 | 2 | 3 | 4        | 5 | 6 | 7 | 8 | 9 |
|--------------|---|---|---|----------|---|---|---|---|---|
| ld r2,0(r1)  | F | D | X | <b>M</b> | W |   |   |   |   |
| add r1,r3,r4 |   | F | D | X        | M | W |   |   |   |
| sub r1,r3,r5 |   |   | F | D        | X | M | W |   |   |
| st r6,0(r1)  |   |   |   | <b>F</b> | D | X | M | W |   |

- **Structural hazard**: resource needed twice in one cycle
  - Example: unified instruction & data cache
  - Solutions?



# Example Structural Hazard

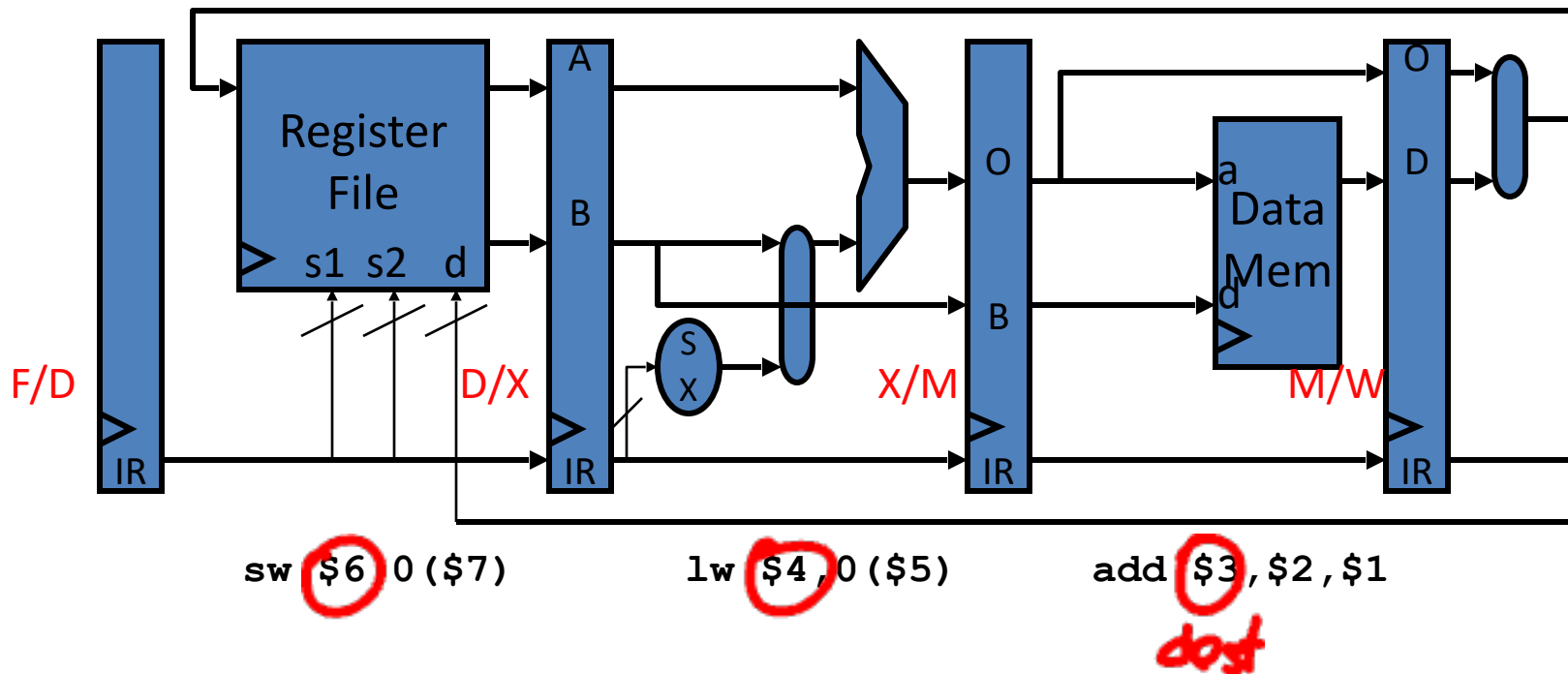
|              | 1 | 2 | 3 | 4        | 5 | 6 | 7 | 8 | 9 |
|--------------|---|---|---|----------|---|---|---|---|---|
| ld r2,0(r1)  | F | D | X | <b>M</b> | W |   |   |   |   |
| add r1,r3,r4 |   | F | D | X        | M | W |   |   |   |
| sub r1,r3,r5 |   |   | F | D        | X | M | W |   |   |
| st r6,0(r1)  |   |   |   | <b>F</b> | D | X | M | W |   |

- **Structural hazard**: resource needed twice in one cycle
  - Example: unified instruction & data cache
  - Solutions?
    - Separate instruction/data caches
    - Redesign cache to allow 2 accesses per cycle (slower, more expensive)
    - Stall pipeline



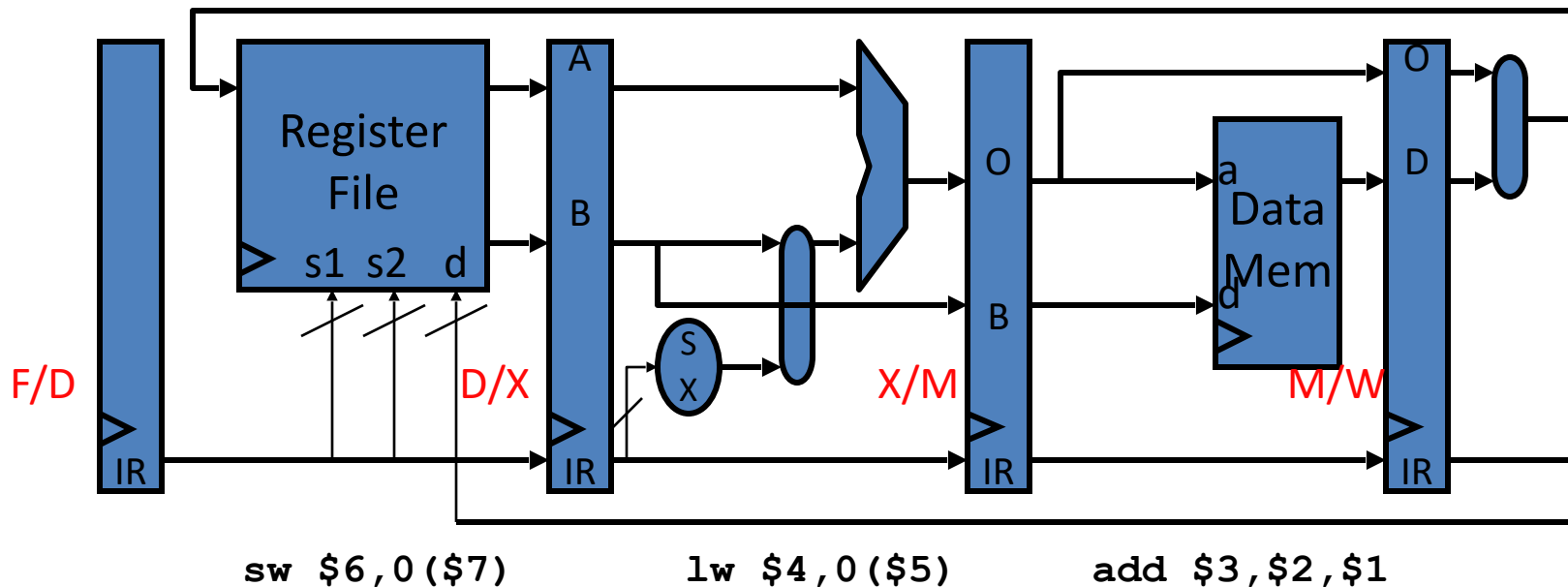
↳ (#ports)²

# Data Hazards



- Let's forget about branches and the control for a while
- The three insn sequence we saw earlier executed fine...
  - Can you imagine situations when it will **not** be fine?

# Data Hazards



- Let's forget about branches and the control for a while
- The three insn sequence we saw earlier executed fine...
  - But it wasn't a real program
  - Real programs have data dependences
    - They pass values via registers and memory

# Dependent Operations

- Independent operations

```
add $3 <- $2, $1  
add $6 <- $5, $4
```

- Would this program execute correctly on the last pipeline ?

```
add $3 <- $2, $1  
add $6 <- $5, $3
```

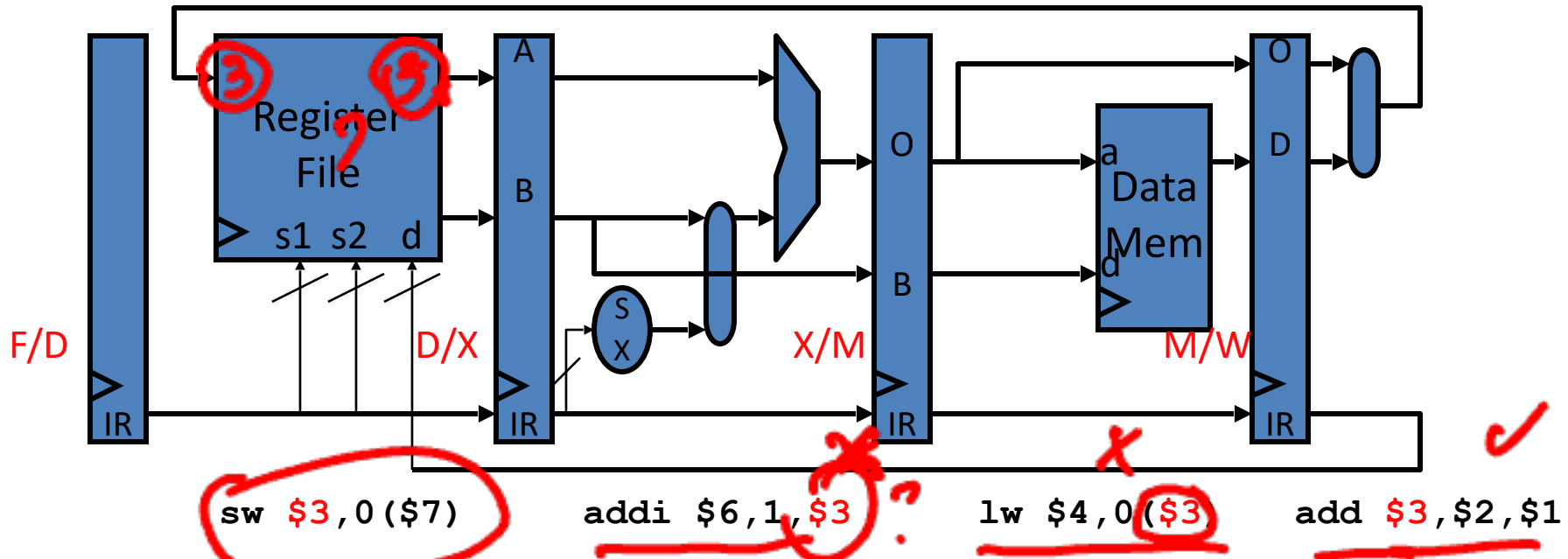
- What about this program?

```
add $3 <- $2, $1  
lw $4 <- 0($3)  
addi $6, 1, $3  
sw $3 -> 0($7)
```

RAW

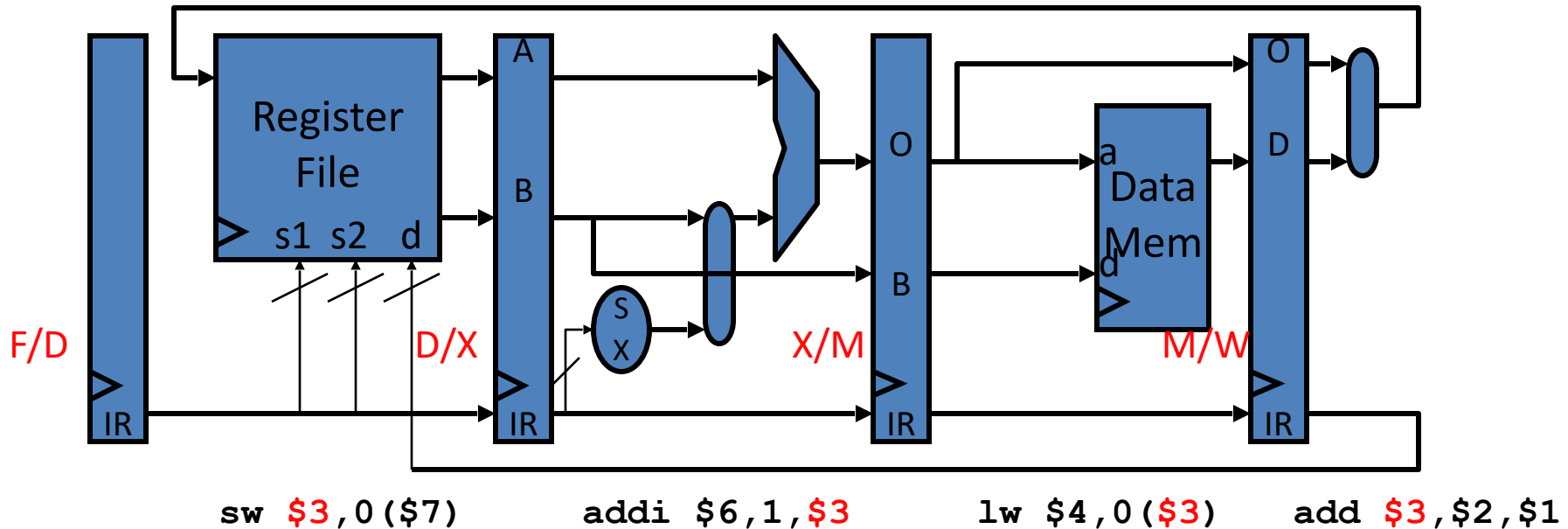
WAL anti-dependence

# Data Hazards



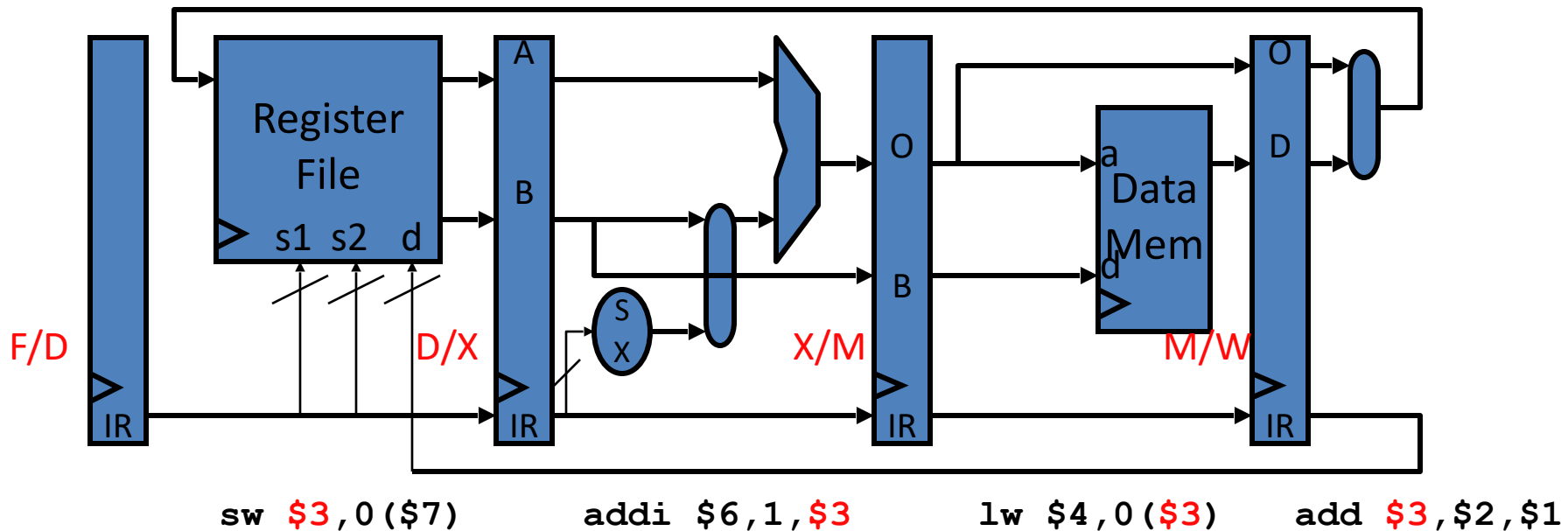
- Would this "program" execute correctly on this pipeline?

# Data Hazards



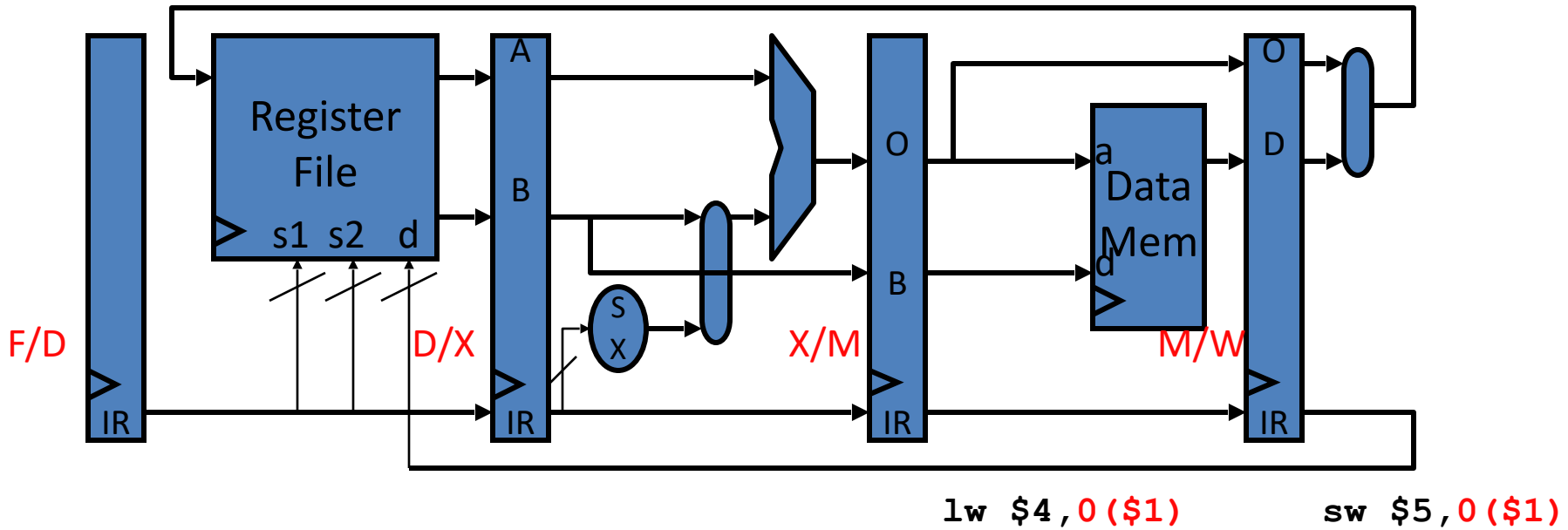
- Would this “program” execute correctly on this pipeline?
  - Which insns would execute with correct inputs?

# Data Hazards



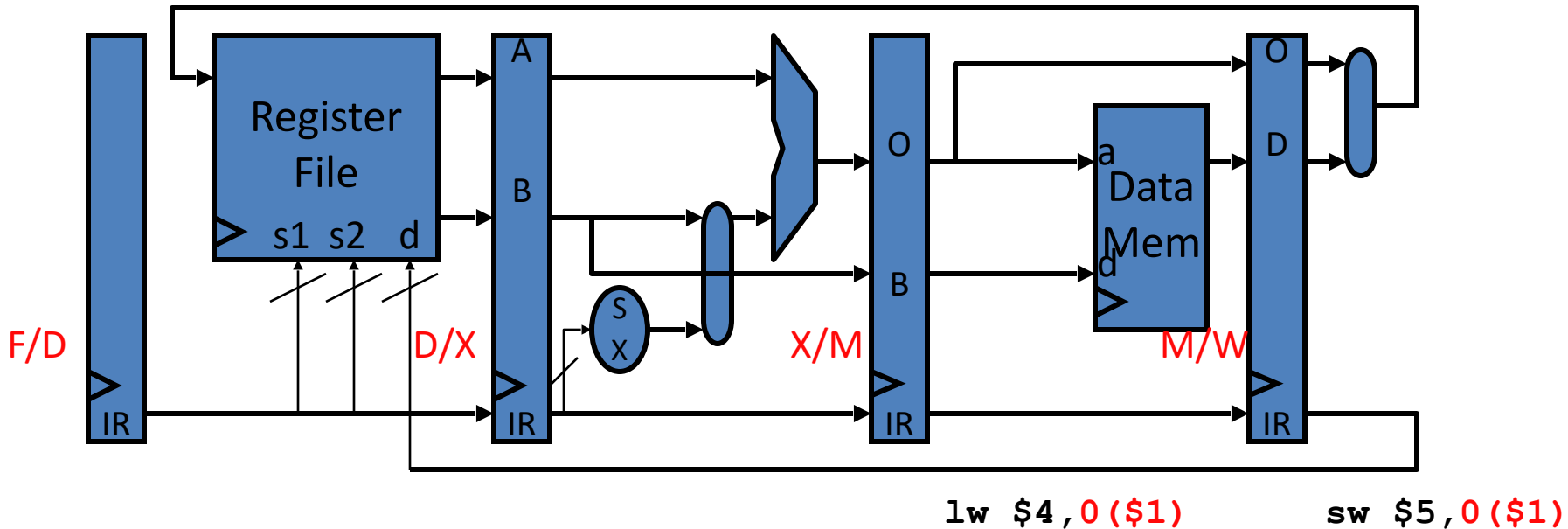
- Would this “program” execute correctly on this pipeline?
  - Which insns would execute with correct inputs?
  - **add** is writing its result into **\$3** in current cycle
  - **lw** read **\$3** 2 cycles ago → got wrong value
  - **addi** read **\$3** 1 cycle ago → got wrong value
  - **sw** is reading **\$3** ~~this cycle~~ → maybe (depending on regfile design)

# Memory Data Hazards



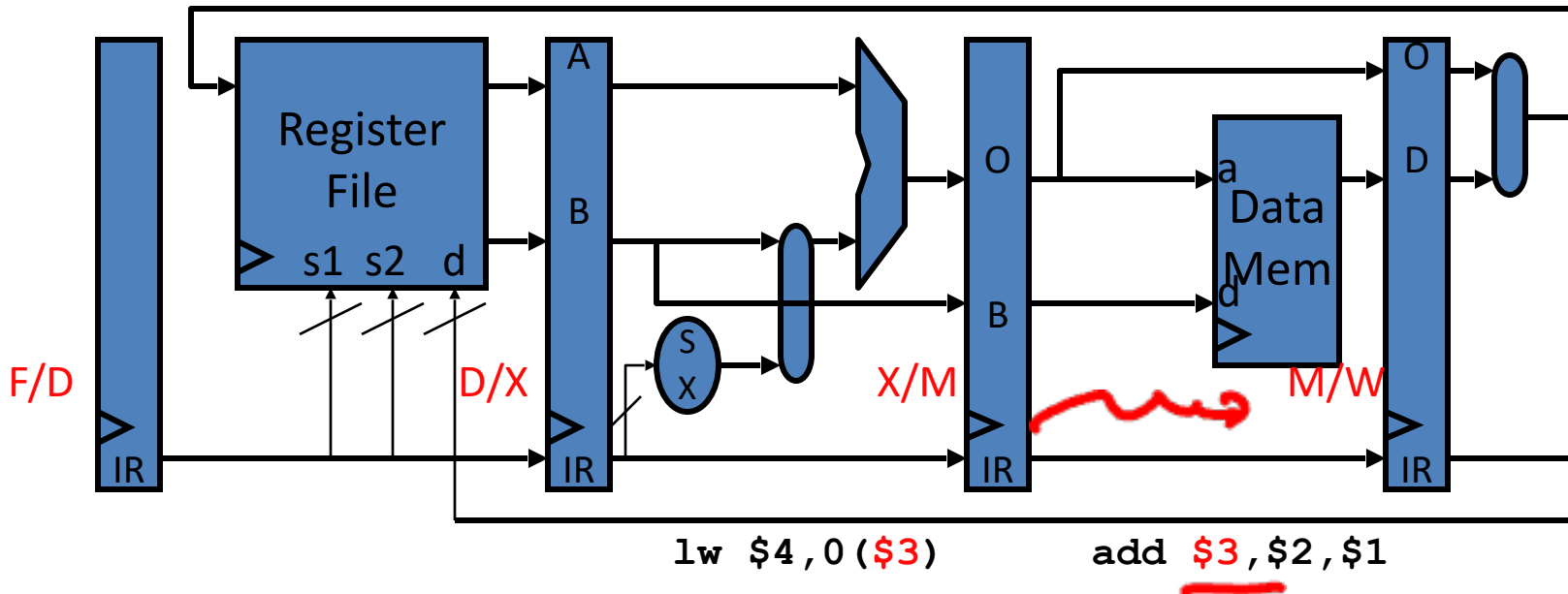
- What about data hazards through memory, is that a hazard?

# Memory Data Hazards



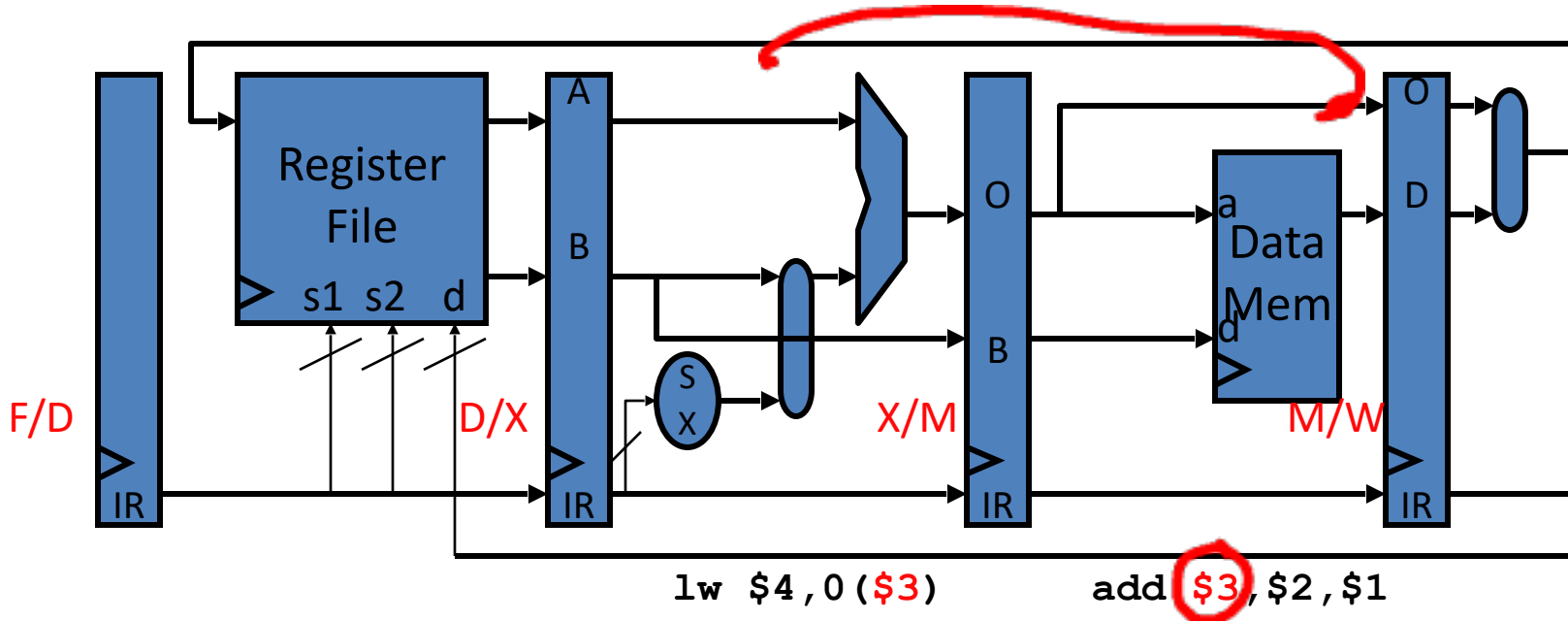
- What about data hazards through memory? No
  - lw following sw to same address in next cycle, gets right value
  - Why? Data mem read/write always take place in same stage

# Observation!



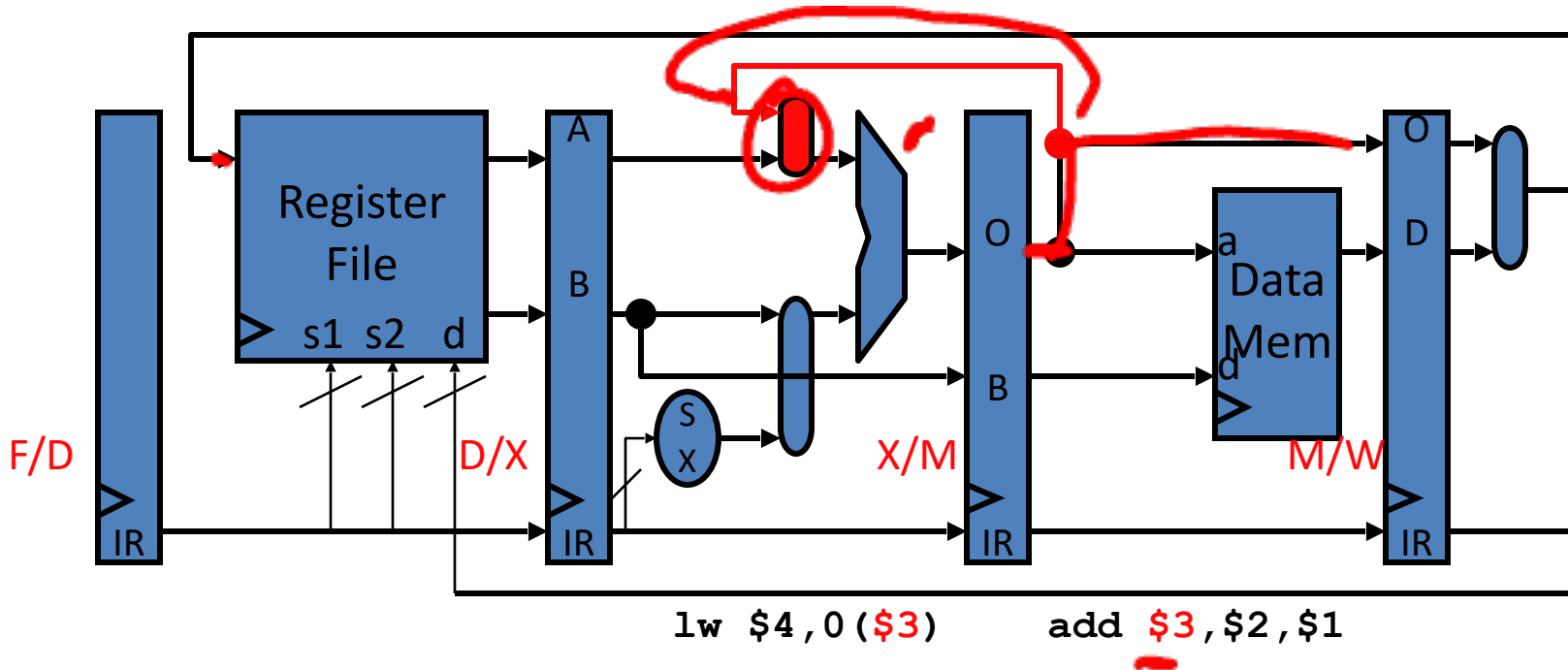
- Technically, this situation is broken
  - `lw $4, 0($3)` has already read `$3` from regfile
  - `add $3, $2, $1` hasn't yet written `$3` to regfile
- But ...😊

# Observation!



- Technically, this situation is broken
  - `lw $4, 0($3)` has already read `$3` from regfile
  - `add $3, $2, $1` hasn't yet written `$3` to regfile
- But fundamentally, everything is OK
  - `lw $4, 0($3)` hasn't actually used `$3` yet
  - `add $3, $2, $1` has already computed `$3`
- *How can we take advantage of this?*

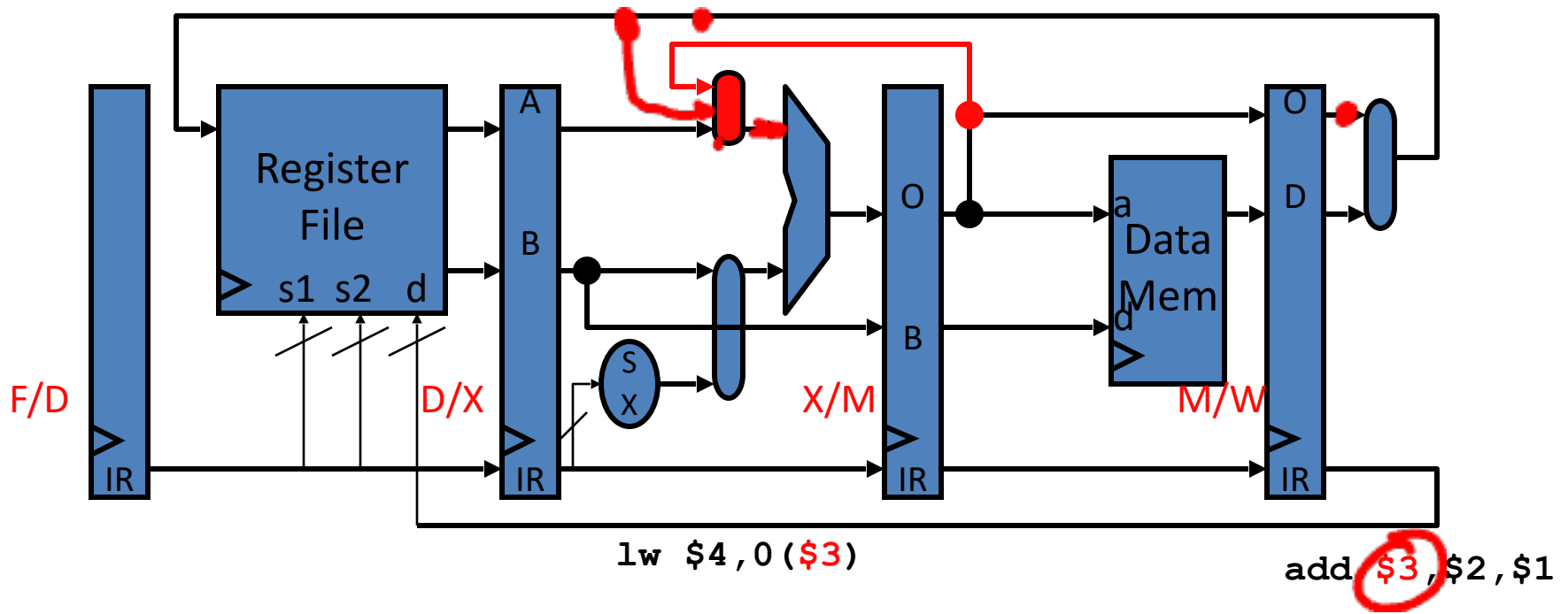
# Reducing Data Hazards: Bypassing



- **Bypassing**

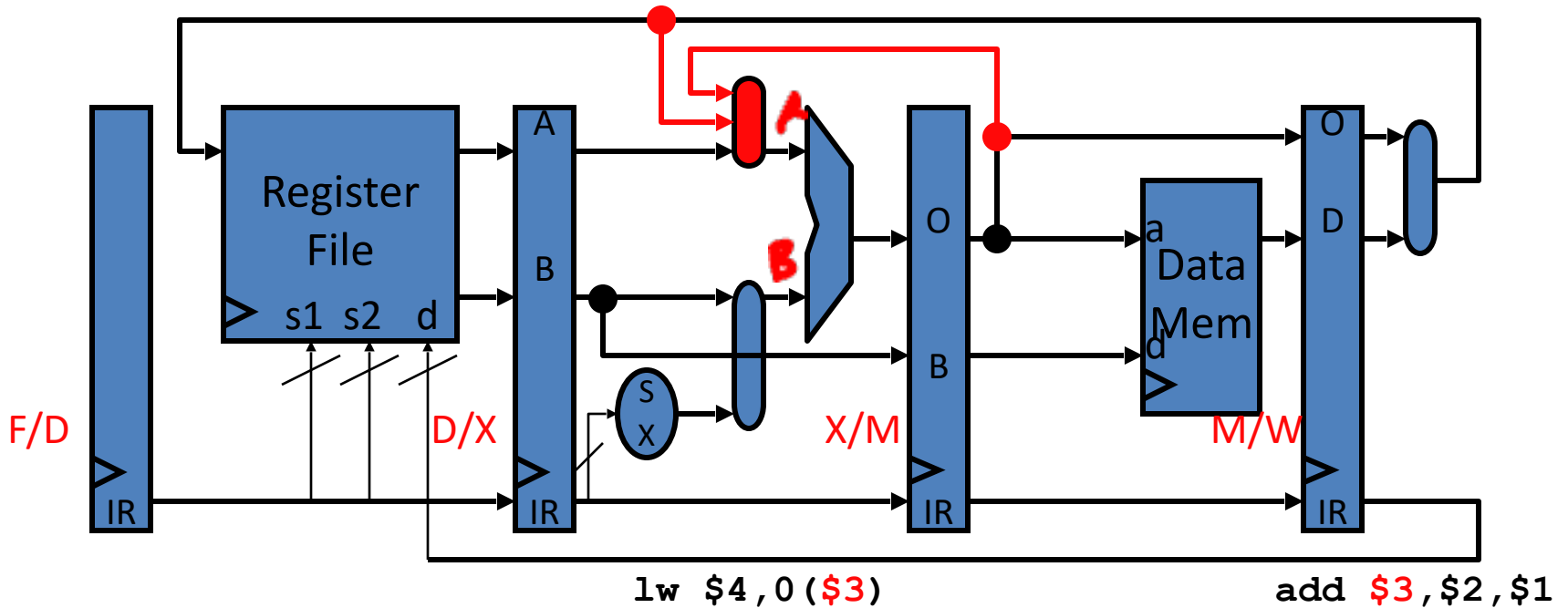
- Reading a value from an intermediate (μarchitectural) source
- Not waiting until it is available from primary source
- Here, we are bypassing the register file
- Also called **forwarding**

# What about this situation?



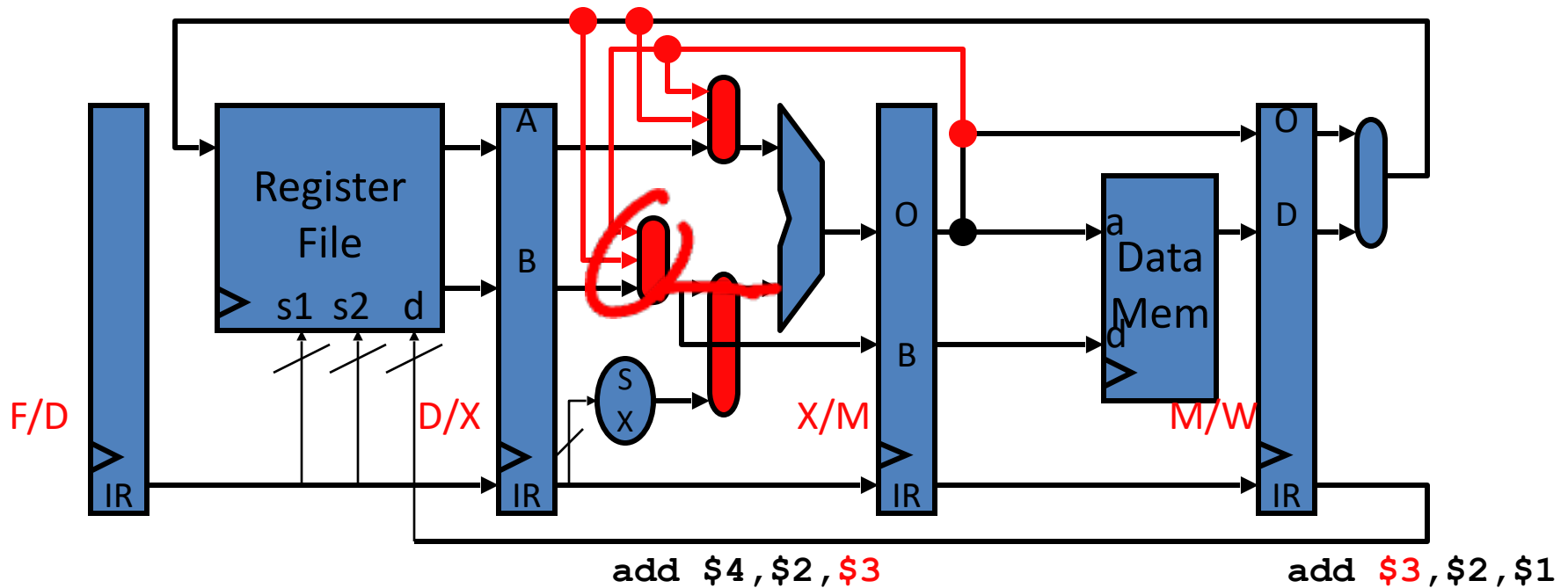
- Would the bypassing above work?

# WX Bypassing



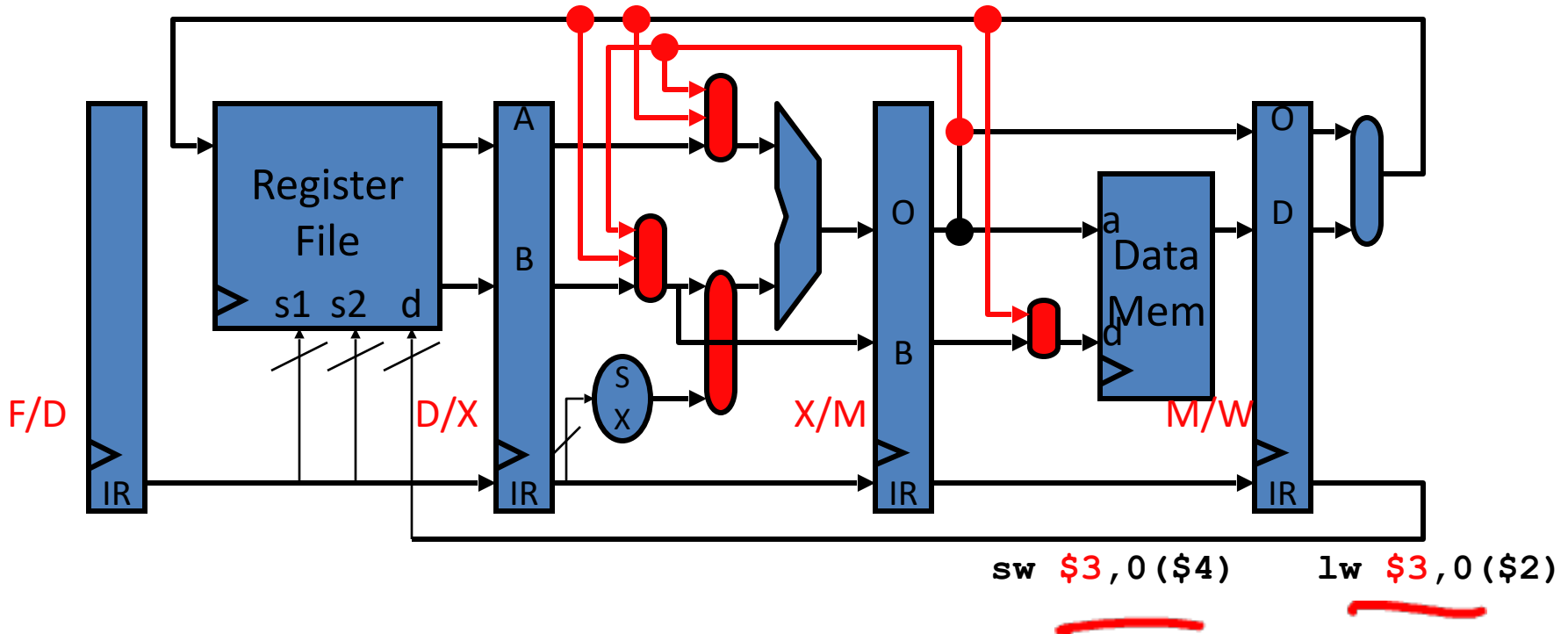
- Add another bypass path and MUX input
- First one was an **MX** bypass
- This one is a **WX** bypass

# ALUinB Bypassing



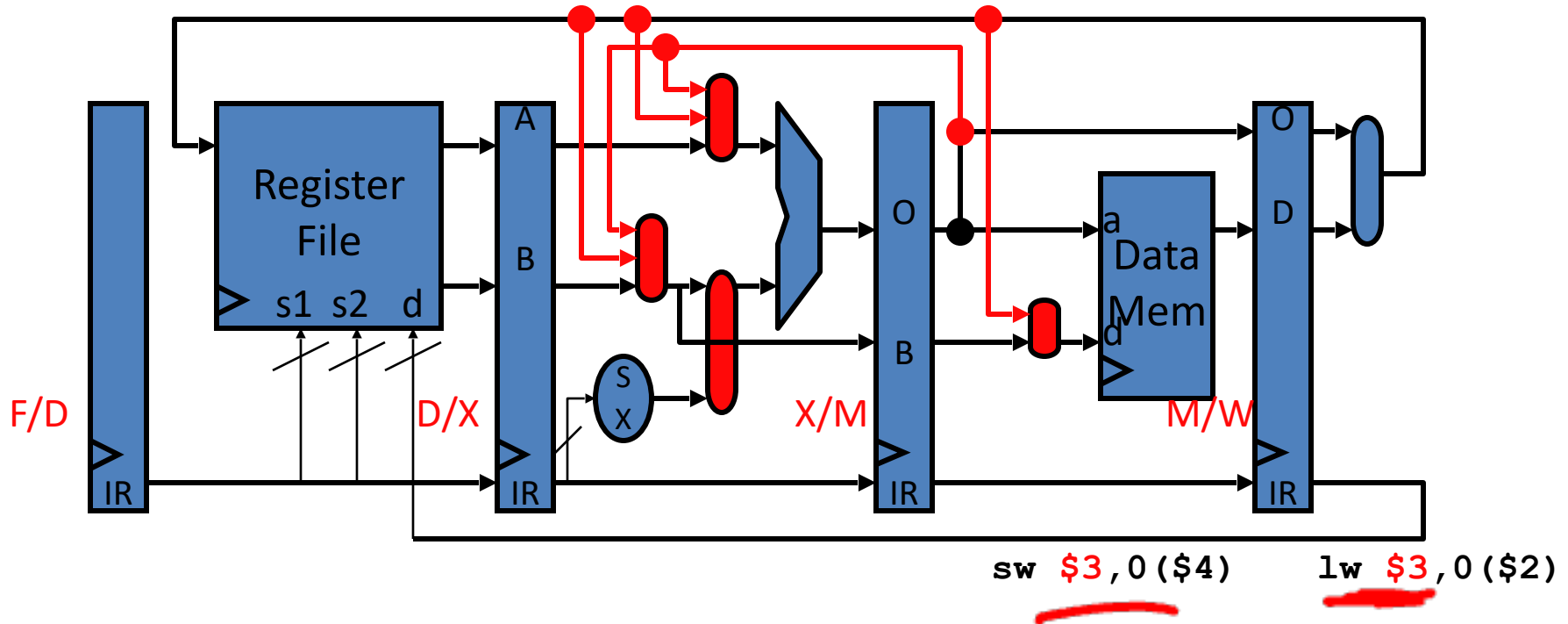
- Can also bypass to ALU input B

# WM Bypassing?



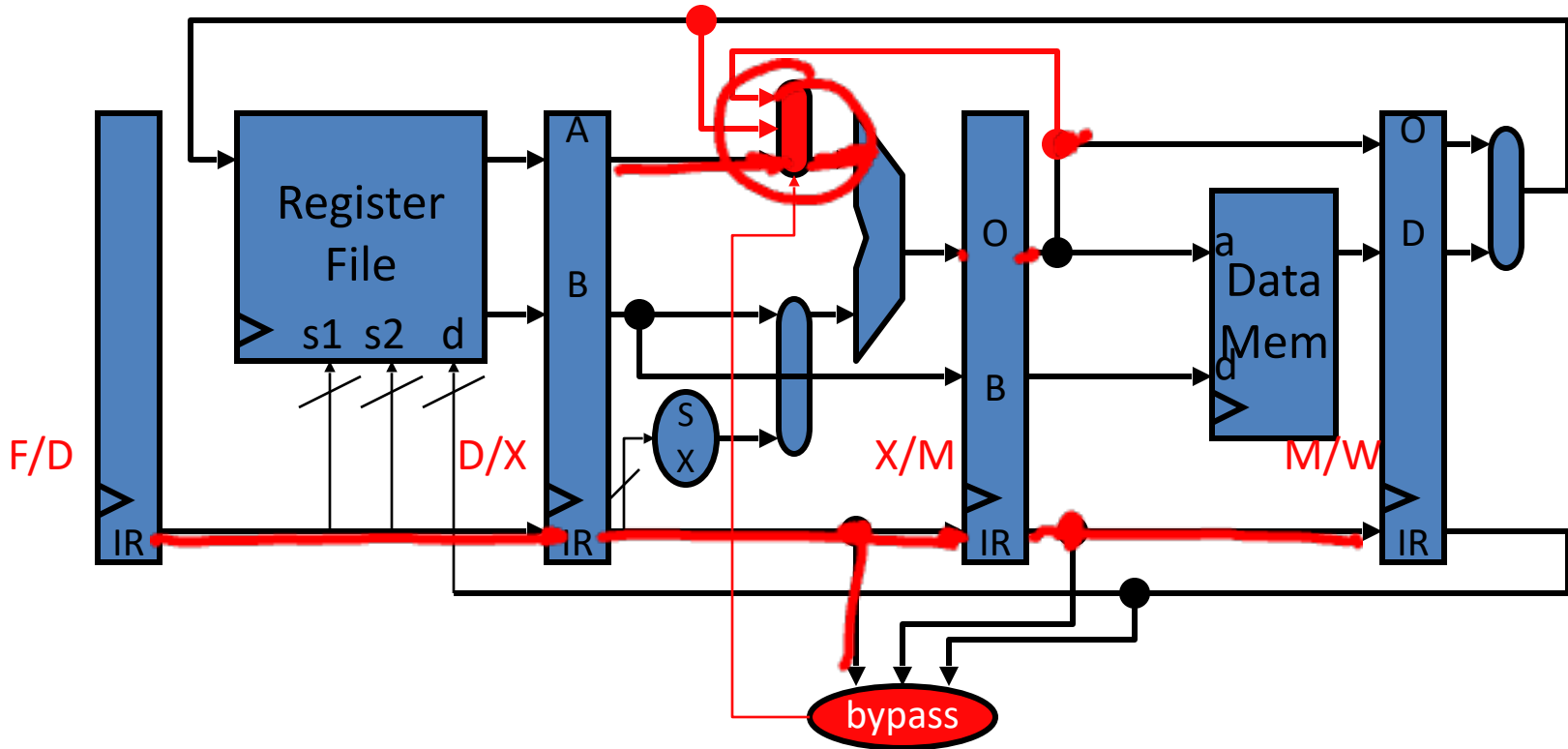
- Does WM bypassing make sense?

# WM Bypassing?



- Does WM bypassing make sense? Memcopy!

# Bypass Logic



- Each MUX has its own, here it is for MUX ALUinA

$(D/X.IR.RegSource1 == X/M.IR.RegDest) \Rightarrow 0$

$(D/X.IR.RegSource1 == M/W.IR.RegDest) \Rightarrow 1$

Else  $\Rightarrow 2$

# Pipeline Diagrams with Bypassing

- If bypass exists, “from”/“to” stages execute in same cycle

– Example: full bypassing, use MX bypass

|                      | 1 | 2 | 3        | 4        | 5 | 6 | 7 | 8 | 9 | 10 |
|----------------------|---|---|----------|----------|---|---|---|---|---|----|
| add r2,r3→ <b>r1</b> | F | D | <b>X</b> | <b>M</b> | W |   |   |   |   |    |
| sub <b>r1</b> ,r4→r2 |   | F | D        | <b>X</b> | M | W |   |   |   |    |

- Example: full bypassing, use WX bypass

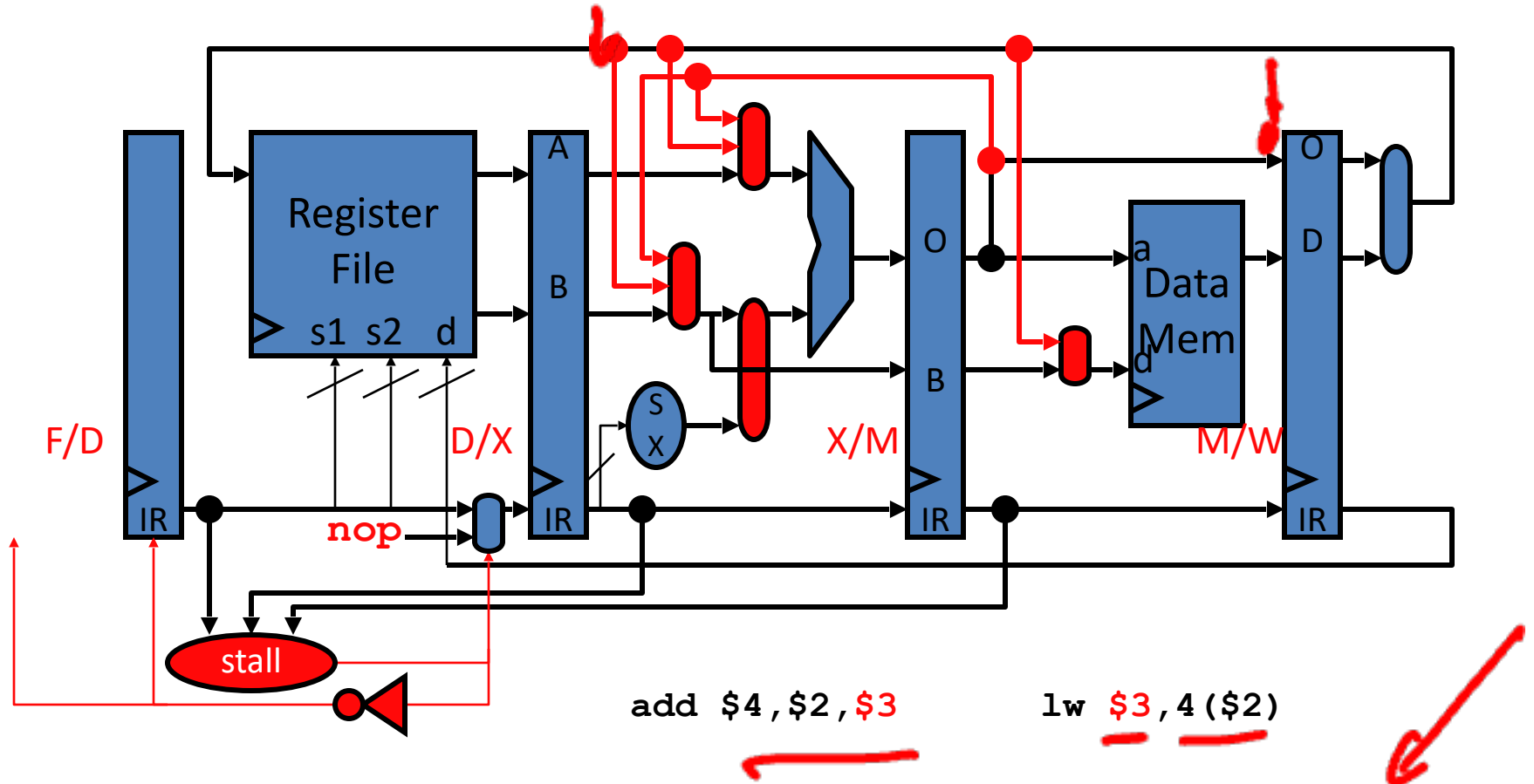
|                      | 1 | 2 | 3 | 4 | 5        | 6 | 7 | 8 | 9 | 10 |
|----------------------|---|---|---|---|----------|---|---|---|---|----|
| add r2,r3→ <b>r1</b> | F | D | X | M | <b>W</b> |   |   |   |   |    |
| ld [r7]→r5           |   | F | D | X | M        | W |   |   |   |    |
| sub <b>r1</b> ,r4→r2 |   |   | F | D | <b>X</b> | M | W |   |   |    |

- Example: WM bypass

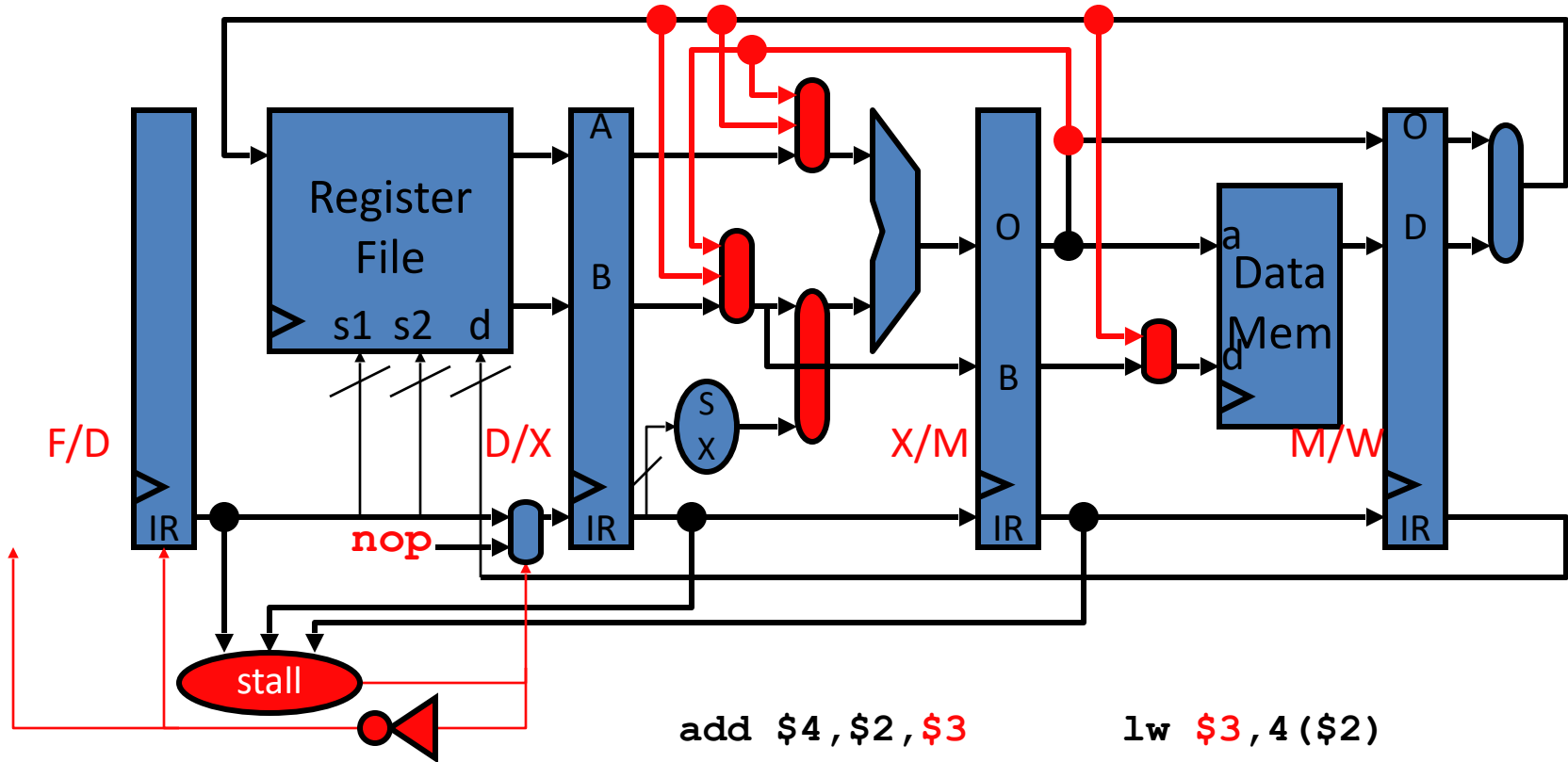
|                                                | 1 | 2 | 3 | 4 | 5        | 6 | 7 | 8 | 9 | 10 |
|------------------------------------------------|---|---|---|---|----------|---|---|---|---|----|
| add r2,r3→ <b>r1</b>                           | F | D | X | M | <b>W</b> |   |   |   |   |    |
| → ? <b>ld r2,r0(r1)</b><br><b>st r1,r3(r3)</b> |   | F | D | X | <b>M</b> | W |   |   |   |    |

- Can you think of a code example that uses the WM bypass?

# Have We Prevented All Data Hazards?

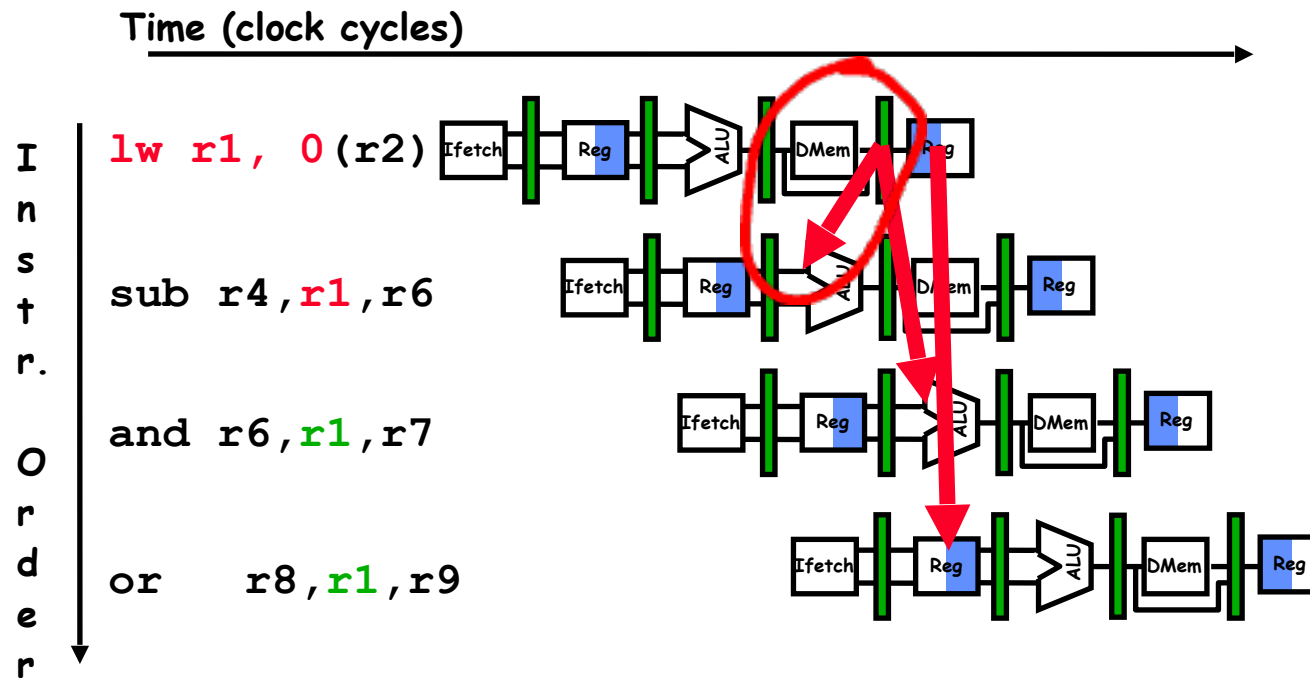


# Have We Prevented All Data Hazards?

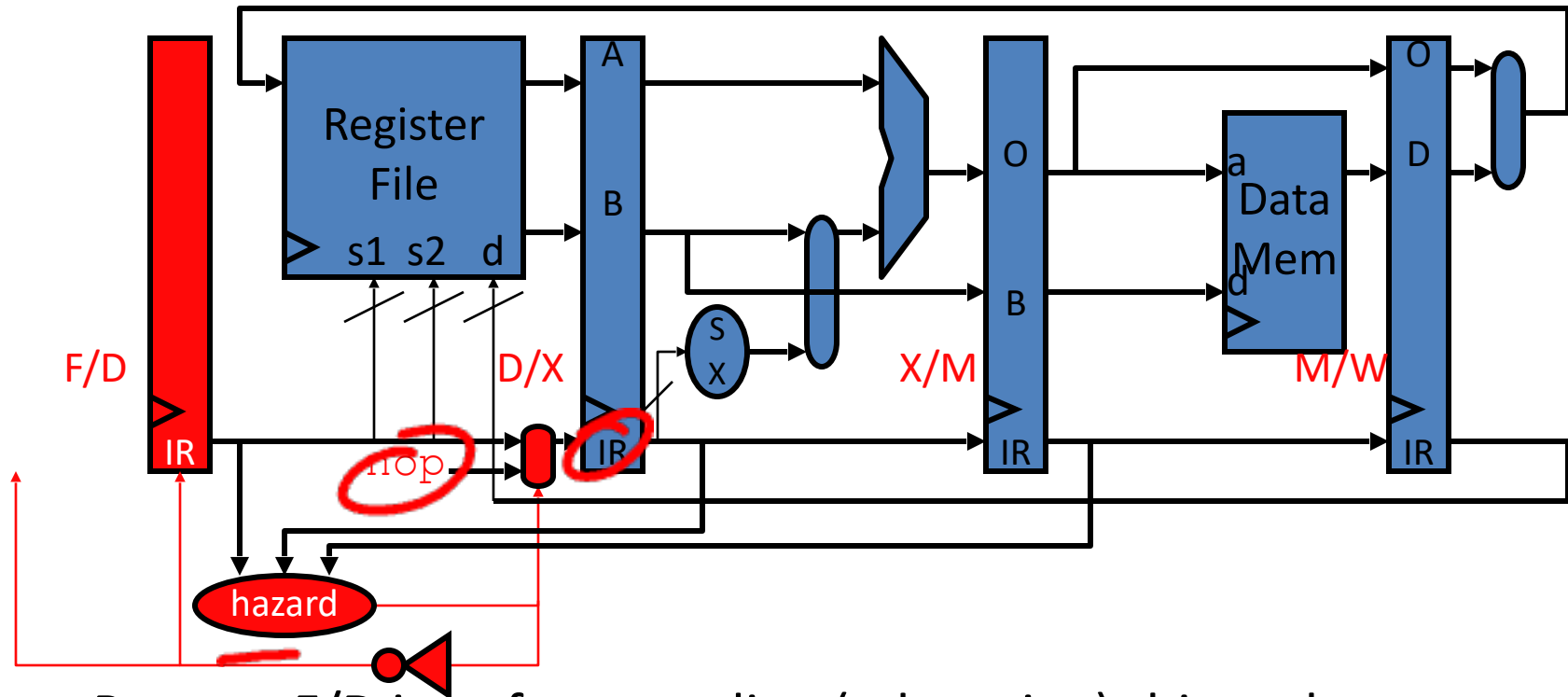


- No. Consider a “load” followed by a dependent “add” insn
- Bypassing alone isn't sufficient
- Solution? Detect this, and then stall the “add” by one cycle

# Loads followed by use



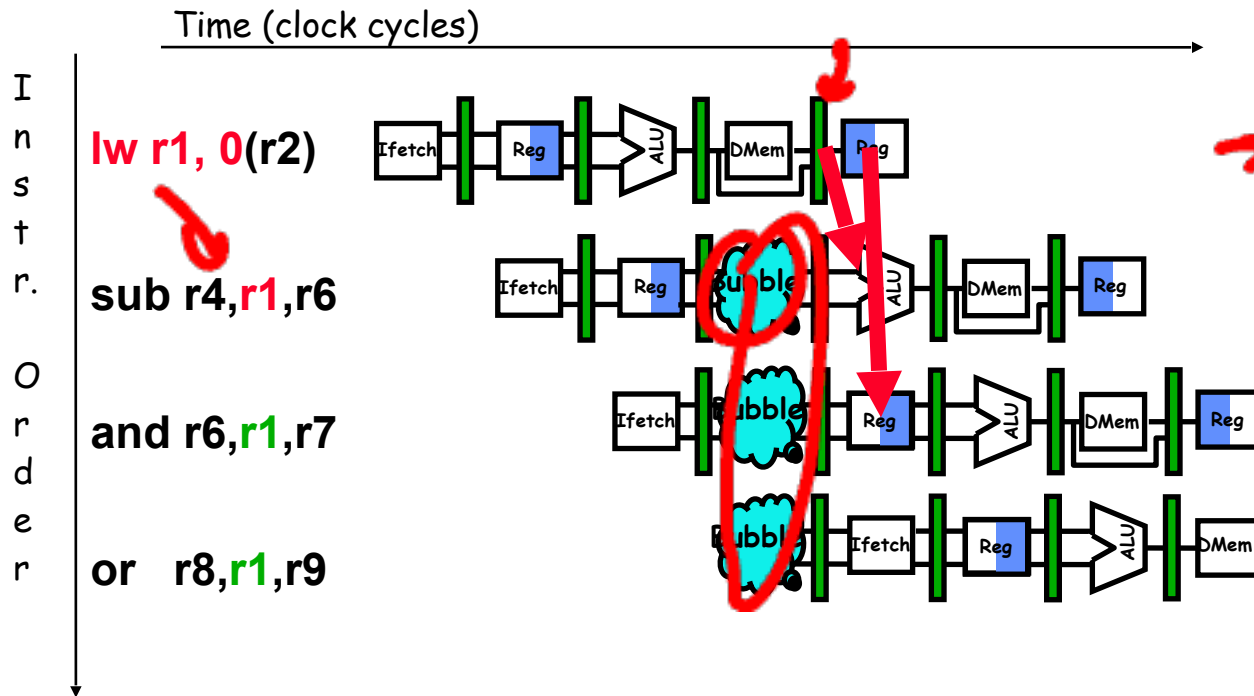
# Stalling to Avoid Data Hazards



- Prevent F/D insn from reading (advancing) this cycle
  - Write **nop** into D/X.IR (effectively, insert **nop** in hardware)
  - Also reset (clear) the datapath control signals
  - Disable F/D latch and PC write enables (why?)
- Re-evaluate situation next cycle

# Loads followed by use stall

look into  
delay  
stall



The ARM1136JF-S processor features:

- an integer unit with integral EmbeddedICE-RT logic
- an eight-stage pipeline
- branch prediction with return stack
- low interrupt latency
- external coprocessor interface and coprocessors 14 and 15
- Instruction and Data *Memory Management Units* (MMUs), managed using MicroTLB structures backed by a unified Main TLB
- Instruction and data caches, including a non-blocking data cache with *Hit-Under-Miss* (HUM)
- the caches are virtually indexed and physically addressed
- 64-bit interface to both caches
- a bypassable write buffer
- level one *Tightly-Coupled Memory* (TCM) that can be used as a local RAM with DMA, or as SmartCache
- high-speed *Advanced Microprocessor Bus Architecture* (AMBA) level two interfaces supporting prioritized multiprocessor implementations
- *Vector Floating-Point* (VFP) coprocessor support
- external coprocessor support
- trace support
- JTAG-based debug.

